

Laboratorio di Informatica 1

Corso di Laurea in Fisica

Federico Nati, Uni. Milano-Bicocca, 2021

Lezione 5, 21 aprile 2021

ARGOMENTI

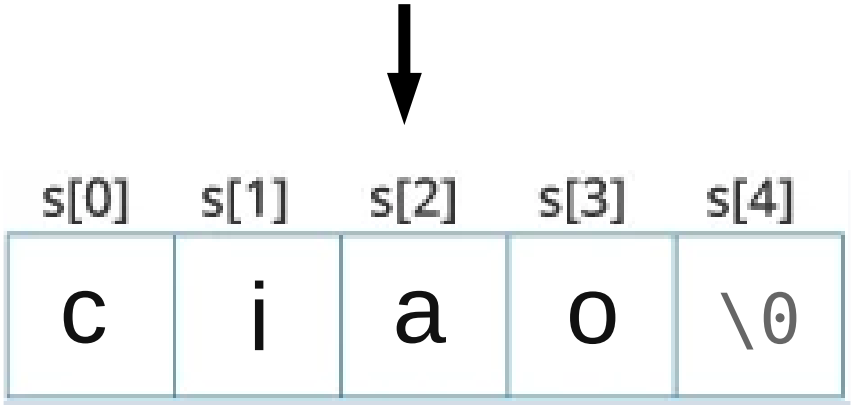
Tra le cose viste nelle scorse lezioni:

- Memoria RAM, valori e indirizzi
- Array e dati strutturati
- Le funzioni
- Rappresentazione di dati in memoria

Oggi vedremo:

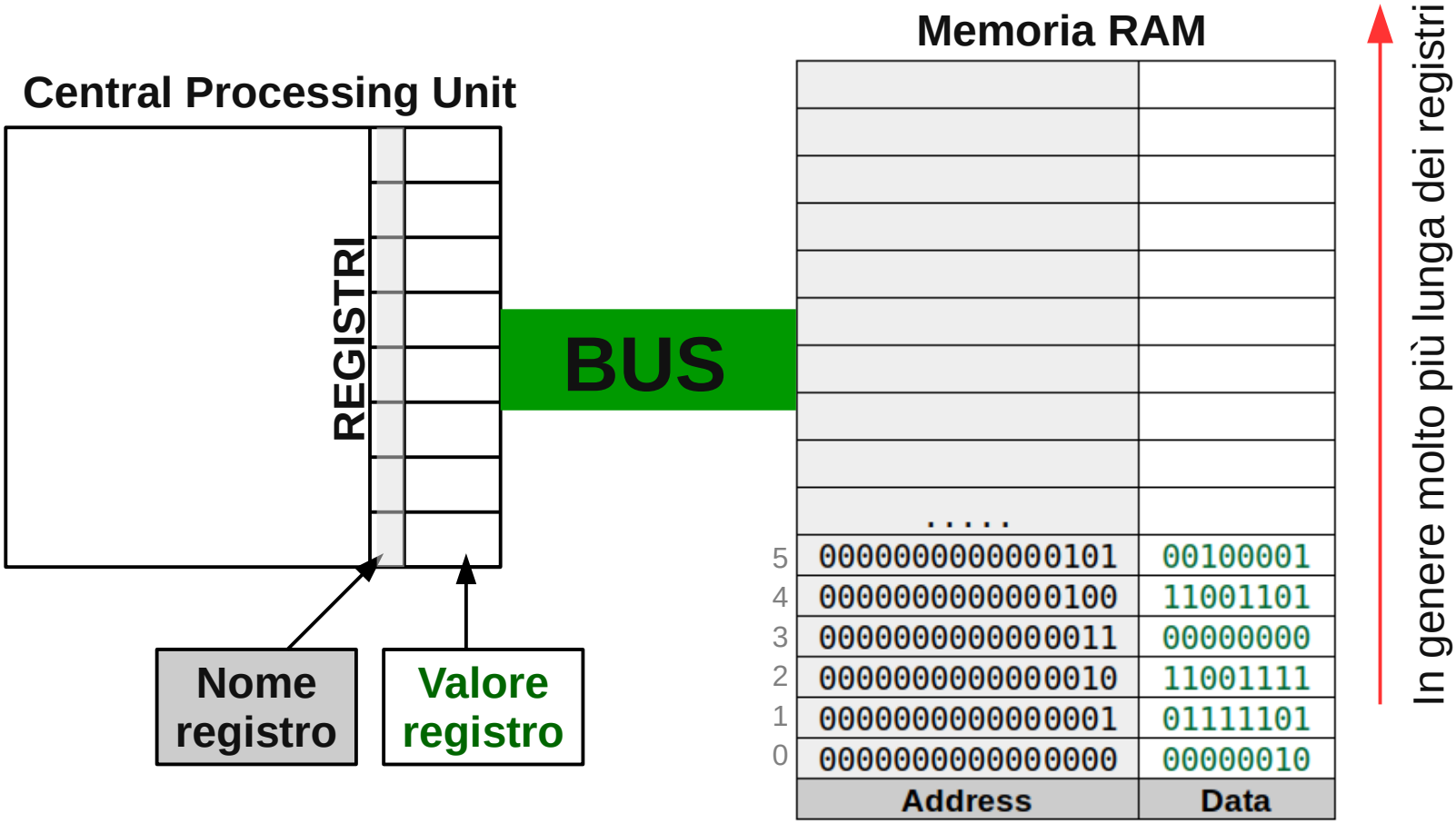
- Come lavorare con gli indirizzi della memoria RAM: i puntatori
- Utilizzo dinamico della memoria RAM: malloc
- Stack (o pile) di dati
- Notazione polacca inversa

```
char s[] = “ciao”;
```



ASCII	
97	a
98	b
99	c
100	d
101	e
102	f
103	g
104	h
105	i
106	j
107	k
108	l
109	m
110	n
111	o
112	p
113	q
114	r
115	s
116	t
117	u
118	v
119	w
120	x
121	y
122	z

ARCHITETTURA DI VON NEUMANN



RAM: VALORI E INDIRIZZI

```
char s[] = “ciao”;
```

	Address	Data		
	.....	...		
5	0000000000000101	...		
4	0000000000000100	00000000	0	\0
3	0000000000000011	01101111	111	o
2	0000000000000010	01100001	97	a
1	0000000000000001	01101001	105	i
0	0000000000000000	01100011	99	c

ASCII	
97	a
98	b
99	c
100	d
101	e
102	f
103	g
104	h
105	i
106	j
107	k
108	l
109	m
110	n
111	o
112	p
113	q
114	r
115	s
116	t
117	u
118	v
119	w
120	x
121	y
122	z

RAM: VALORI E INDIRIZZI

```
char s[] = “ciao”;
```

Identificatore (nome)

	Address	Data		
	.....	...		
5	0000000000000101	...		
4	0000000000000100	00000000	0	\0
3	0000000000000011	01101111	111	o
2	0000000000000010	01100001	97	a
1	0000000000000001	01101001	105	i
0	0000000000000000	01100011	99	c

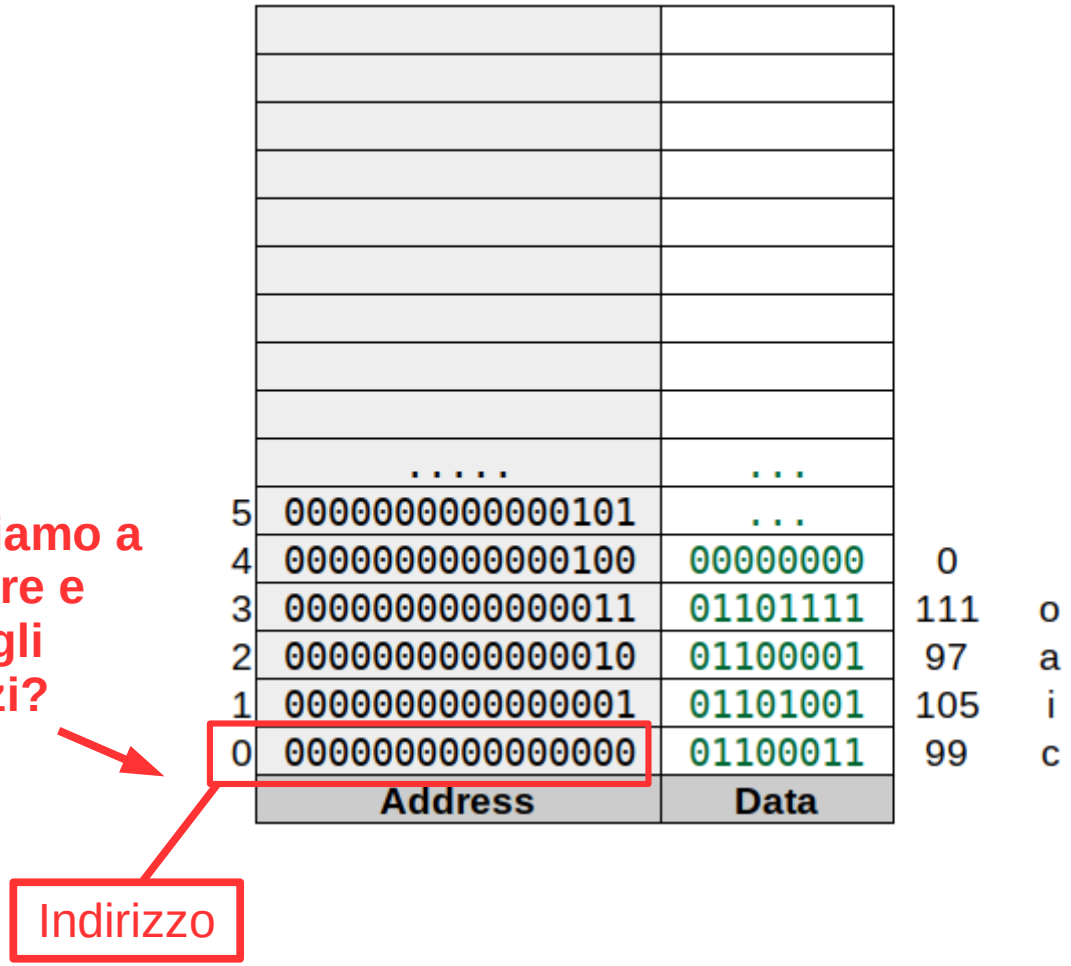
Indirizzo

Valore

ASCII	
97	a
98	b
99	c
100	d
101	e
102	f
103	g
104	h
105	i
106	j
107	k
108	l
109	m
110	n
111	o
112	p
113	q
114	r
115	s
116	t
117	u
118	v
119	w
120	x
121	y
122	z

RAM: VALORI E INDIRIZZI

Come facciamo a conoscere e usare gli indirizzi?



PUNTATORI

- Vorremmo delle “variabili” che contengono indirizzi di memoria
- Il valore della variabile dovrebbe essere la rappresentazione di una locazione di memoria
- In C, questo tipo di variabili si chiamano **puntatori**
- Devo definire opportuni operatori per dichiarare e utilizzare tali puntatori

DICHIARAZIONE DI UN PUNTATORE

Tipo puntato

Indica il tipo di contenuto della locazione di memoria di cui abbiamo l'indirizzo.
Possono essere interi, double, strutture, etc.

Asterisco

Indica che il tipo della variabile è puntatore a "tipo puntato"

```
tipo_puntato * nome_variabile;
```

```
int * a;
```
Puntatore a intero

```
float * b;
```
Puntatore a float

```
struct punto * c;
```
Puntatore a struttura (di tipo struct punto)

UTILIZZARE I PUNTATORI

Per usare i puntatori dobbiamo saper fare innanzitutto le seguenti cose:

- Scrivere o leggere **un indirizzo** di memoria nel puntatore
- Scrivere o leggere **il valore** puntato dal puntatore

OTTENERE L'INDIRIZZO

Supponiamo di avere questa variabile:

```
int a = 5;
```

Come otteniamo la sua posizione (indirizzo) in memoria?

`&a` Significa: "l'indirizzo di memoria della variabile *a*"

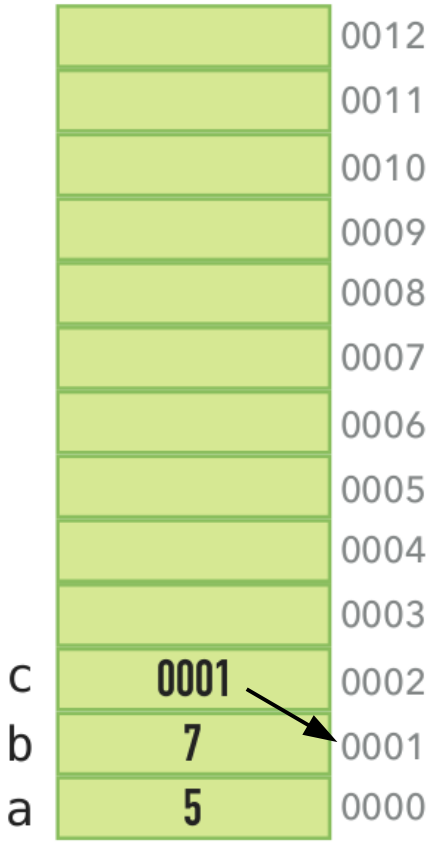
Leggete & come "l'indirizzo di memoria di"

RAPPRESENTAZIONE IN MEMORIA

Il valore contenuto nel puntatore è l'indirizzo di memoria in cui c'è "b"

```
int a = 5;
int b = 7;
int * c = &b;
```

Il tipo è "puntatore a int", non semplicemente "int"



OTTENERE L'INDIRIZZO

SÌ

Tutto ciò che ha una posizione ben definita in memoria

```
int a;
int * p = &a;

int a[10];
int * p = &a[5];
```

NO

Tutto quello che non ha una locazione di memoria associata.
e.g., numeri, espressioni

```
int * p = &5;

int n = 5
int * p = &(n + 1);
```

ACCEDERE AL VALORE, DATO L'INDIRIZZO

Per accedere al valore contenuto nella locazione di memoria puntata si esegue una dereferenziazione, ovvero si "segue" il puntatore.

```
int a = 5;
int * p = &a;
printf("%d\n", *p);
```

Dereferenziazione
mettere un asterisco prima del puntatore significa che bisogna prendere il valore *puntato*.

Operatori:
- Indirizzo: **&**
- Risoluzione: *****

Nota: Abbiamo incontrato il simbolo * in C con tre usi diversi: moltiplicazione, dichiarazione di un puntatore, operatore di risoluzione di un puntatore

```
int a = 5;
int * p = &a;
int b = *p + 1;
printf("%d\n", b);
a = 4;
*p = 5;
printf("%d\n", *p);
```

```
int a = 5;
int * p = &a;
int b = *p + 1;
printf("%d\n", b);
a = 4;
*p = 5;
printf("%d\n", *p);
```



```
int a = 5;
int * p = &a;
int b = *p + 1;
printf("%d\n", b);
a = 4;
*p = 5;
printf("%d\n", *p);
```



```
int a = 5;
int * p = &a;
int b = *p + 1;
printf("%d\n", b);
a = 4;
*p = 5;
printf("%d\n", *p);
```



```
int a = 5;
int * p = &a;
int b = *p + 1;
printf("%d\n", b);
a = 4;
*p = 5;
printf("%d\n", *p);
```





```
int a = 5;
int * p = &a;
int b = *p + 1;
printf("%d\n", b);
a = 4;
*p = 5;
printf("%d\n", *p);
```

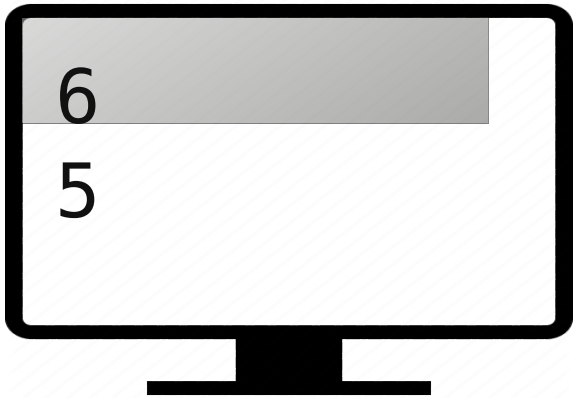


```
int a = 5;
int * p = &a;
int b = *p + 1;
printf("%d\n", b);
a = 4;
*p = 5;
printf("%d\n", *p);
```



```
int a = 5;
int * p = &a;
int b = *p + 1;
printf("%d\n", b);
a = 4;
*p = 5;
printf("%d\n", *p);
```





Uno degli usi per cui i puntatori in C sono fondamentali è il loro passaggio come argomenti a una funzione

Quando vengono passati a una funzione, succede qualcosa di simile a quanto abbiamo visto succedere con il passaggio di array

Il puntatore viene copiato, cioè viene copiato l'indirizzo a cui esso punta

Questo significa che la funzione chiamata fa riferimento alla stessa locazione di memoria della funzione chiamante

In questo modo si passano gli argomenti “per riferimento” (by reference), invece che “per valore” (“by value”)

PUNTATORI E FUNZIONI, UN ESEMPIO

```
#include <stdio.h>
#include <stdlib.h>

void scambio(int * a, int * b)
{
  int tmp = *a;
  *a = *b;
  *b = tmp;
}

int main (int argc, char * argv[])
{
  int x = 4;
  int y = 3;
  scambio(&x, &y);
  exit(EXIT_SUCCESS);
}
```

PUNTATORI E FUNZIONI, UN ESEMPIO

```
#include <stdio.h>
#include <stdlib.h>

void scambio(int * a, int * b)
{
  int tmp = *a;
  *a = *b;
  *b = tmp;
}

int main (int argc, char * argv[])
{
  int x = 4;
  int y = 3;
  scambio(&x, &y);
  exit(EXIT_SUCCESS);
}
```

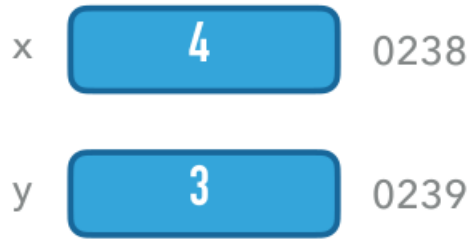


PUNTATORI E FUNZIONI, UN ESEMPIO

```
#include <stdio.h>
#include <stdlib.h>

void scambio(int * a, int * b)
{
  int tmp = *a;
  *a = *b;
  *b = tmp;
}

int main (int argc, char * argv[])
{
  int x = 4;
  int y = 3;
  scambio(&x, &y);
  exit(EXIT_SUCCESS);
}
```

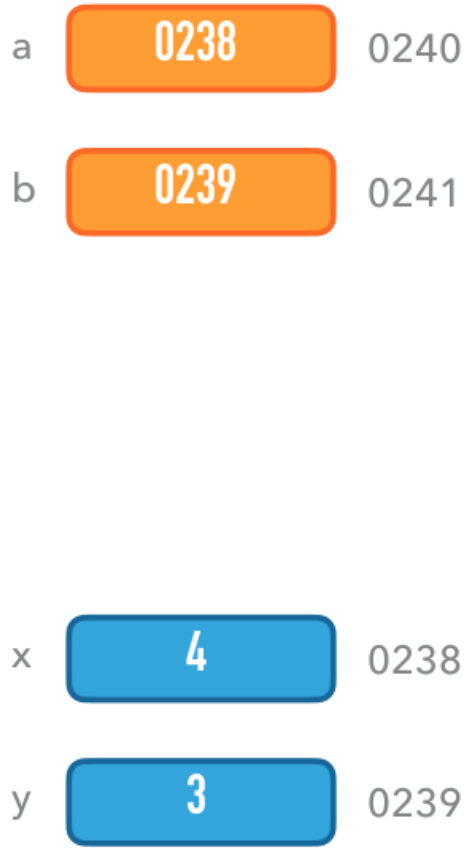


PUNTATORI E FUNZIONI, UN ESEMPIO

```
#include <stdio.h>
#include <stdlib.h>

void scambio(int * a, int * b)
{
  int tmp = *a;
  *a = *b;
  *b = tmp;
}

int main (int argc, char * argv[])
{
  int x = 4;
  int y = 3;
  scambio(&x, &y);
  exit(EXIT_SUCCESS);
}
```

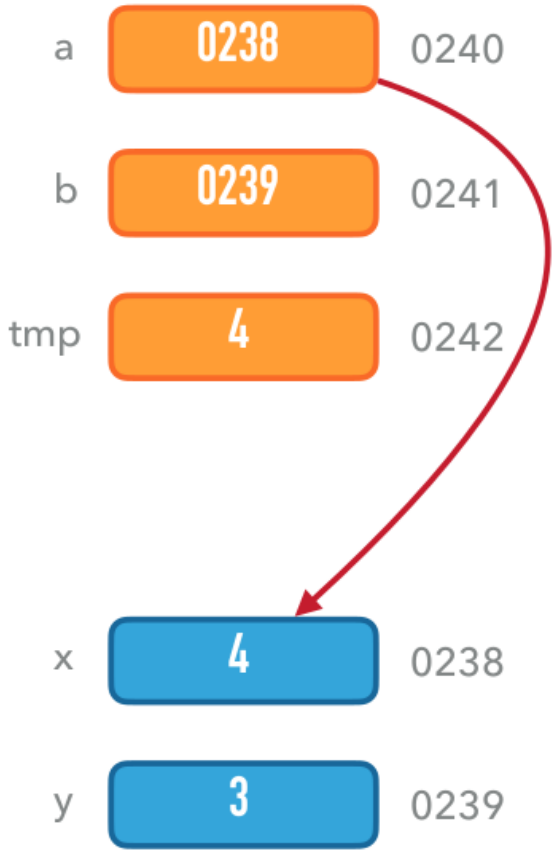


PUNTATORI E FUZIONI, UN ESEMPIO

```c
#include <stdio.h>
#include <stdlib.h>

void scambio(int * a, int * b)
{
  int tmp = *a;
  *a = *b;
  *b = tmp;
}

int main (int argc, char * argv[])
{
  int x = 4;
  int y = 3;
  scambio(&x, &y);
  exit(EXIT_SUCCESS);
}
```

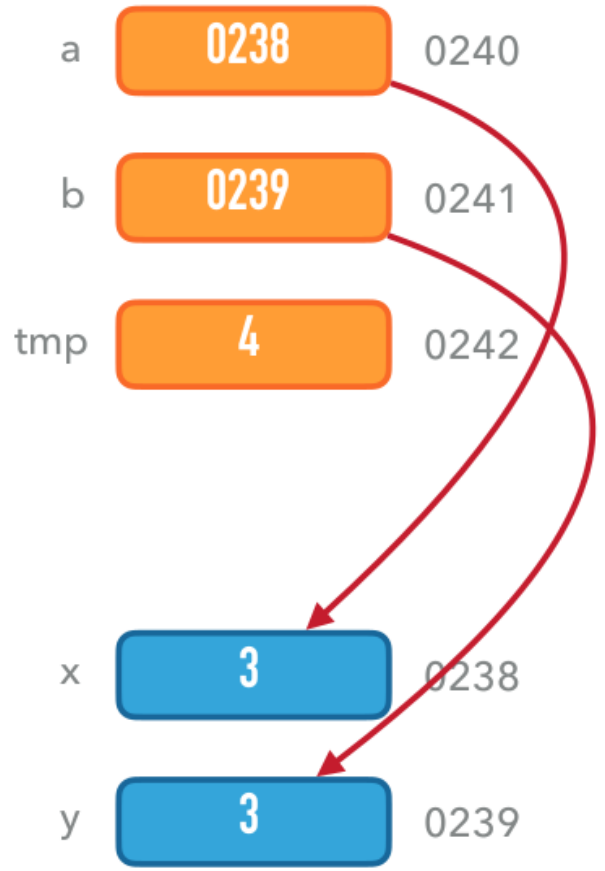


PUNTATORI E FUZIONI, UN ESEMPIO

```c
#include <stdio.h>
#include <stdlib.h>

void scambio(int * a, int * b)
{
  int tmp = *a;
  *a = *b;
  *b = tmp;
}

int main (int argc, char * argv[])
{
  int x = 4;
  int y = 3;
  scambio(&x, &y);
  exit(EXIT_SUCCESS);
}
```

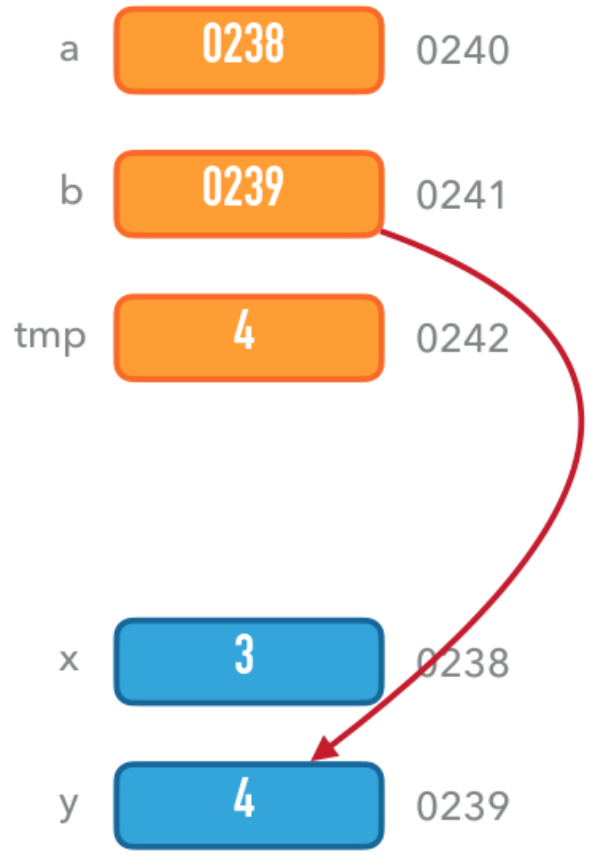


PUNTATORI E FUZIONI, UN ESEMPIO

```c
#include <stdio.h>
#include <stdlib.h>

void scambio(int * a, int * b)
{
  int tmp = *a;
  *a = *b;
  *b = tmp;
}

int main (int argc, char * argv[])
{
  int x = 4;
  int y = 3;
  scambio(&x, &y);
  exit(EXIT_SUCCESS);
}
```

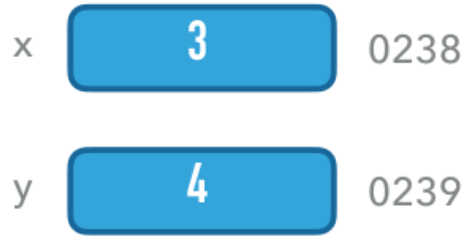


PUNTATORI E FUZIONI, UN ESEMPIO

```
#include <stdio.h>
#include <stdlib.h>

void scambio(int * a, int * b)
{
  int tmp = *a;
  *a = *b;
  *b = tmp;
}

int main (int argc, char * argv[])
{
  int x = 4;
  int y = 3;
  scambio(&x, &y);
  exit(EXIT_SUCCESS);
}
```



- Sembra chiaro a questo punto che i puntatori hanno molto in comune con gli array, e che gli array sono trattabili in molti casi (non tutti) come puntatori

- Abbiamo visto infatti che non è possibile, in C, restituire un array attraverso il return di una funzione

- Tuttavia, anche se le funzioni non possono restituire array, esse possono restituire puntatori

Si possono usare puntatori a strutture, tuttavia per accedere ai singoli campi si deve dereferenziare il puntatore, per esempio:

```
struct punto a;
a.x = 4;
a.y = 5;
struct punto * p = &a;
```

~~p.x~~ ~~*(p.x)~~ (*p).x che può anche essere scritto p->x

```c
#include <stdio.h>
#include <stdlib.h>

struct punto {
  float x;
  float y;
};

void riempi(struct punto * p)
{
  p->x = 2;
  p->y = 4;
}

int main (int argc, char * argv[])
{
  struct punto a;
  riempi(&a);
  exit(EXIT_SUCCESS);
}
```



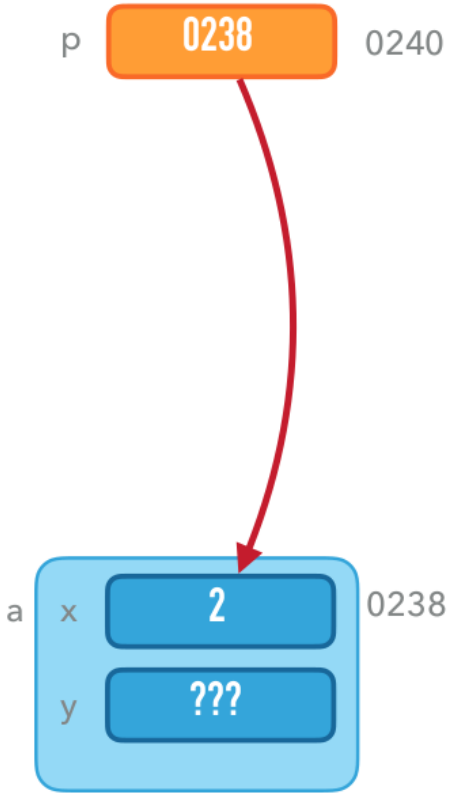


```c
#include <stdio.h>
#include <stdlib.h>

struct punto {
  float x;
  float y;
};

void riempi(struct punto * p)
{
  p->x = 2;
  p->y = 4;
}

int main (int argc, char * argv[])
{
  struct punto a;
  riempi(&a);
  exit(EXIT_SUCCESS);
}
```

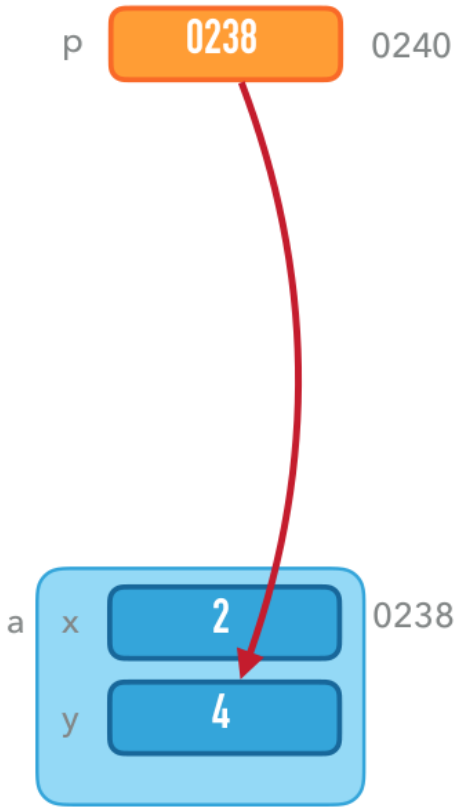


```c
#include <stdio.h>
#include <stdlib.h>

struct punto {
  float x;
  float y;
};

void riempi(struct punto * p)
{
  p->x = 2;
  p->y = 4;
}

int main (int argc, char * argv[])
{
  struct punto a;
  riempi(&a);
  exit(EXIT_SUCCESS);
}
```



ERRORI CON I PUNTATORI

```
int a = 3;
int b = 4;
int * p = &b;
*p = &a;
```

Assegnare un indirizzo alla memoria puntata e non al puntatore

```
int a = 3;
int * p;
p = a;
```

Assegnare un valore che non rappresenta un indirizzo al puntatore e non alla memoria puntata

```
int a = 3;
int * p;
a = *p + 1;
```

Dereferenziare un puntatore non inizializzato. Che indirizzo contiene?

```
int * p;
*p = 5;
```

Assegnare un valore alla cella di memoria puntata da un puntatore non inizializzato. Dove viene messo il valore?

```
int * ritorna_puntatore()
{
  int x = 42;
  int * p = &x;
  return p;
}
```

Restituire la locazione di memoria di una variabile locale di una funzione, che cessa di esistere dopo il return

...tutti errori comuni da evitare!

UNA RIFLESSIONE SUL TIPO DEL PUNTATORE

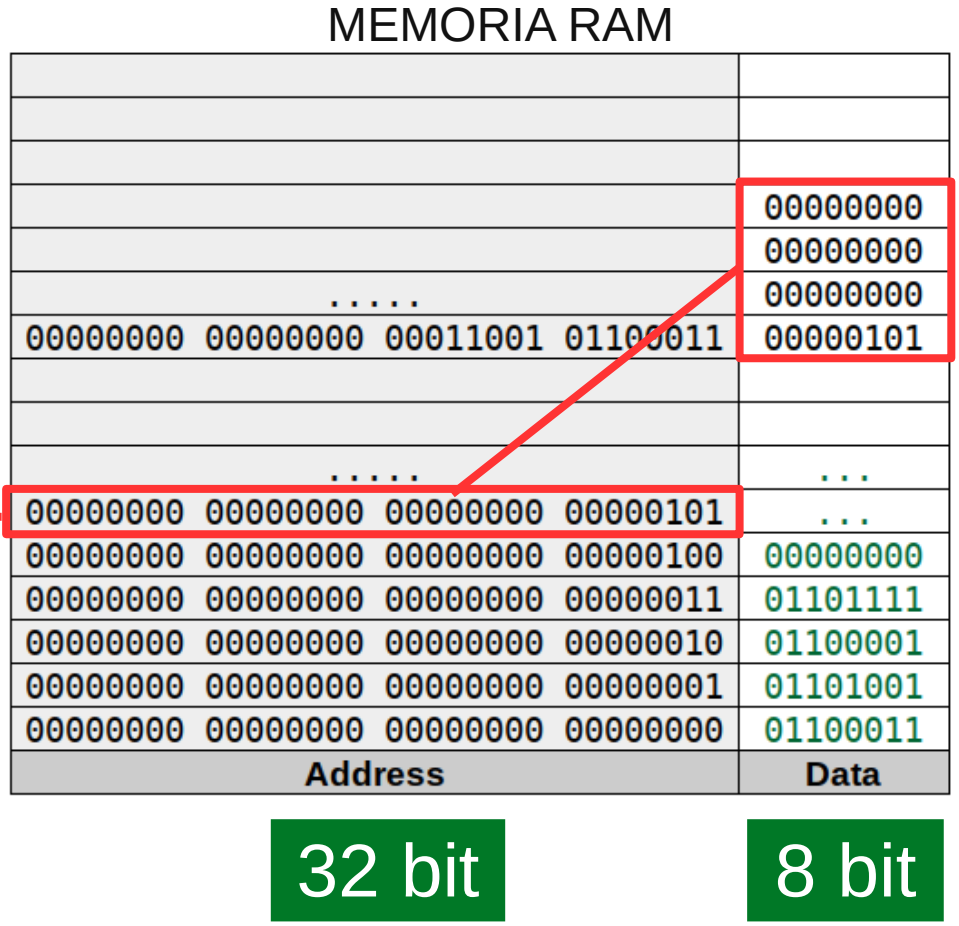
Il puntatore è una variabile il cui valore è un indirizzo di memoria

In un sistema il cui processore utilizza, per esempio, 32 bit per rappresentare un indirizzo di memoria nei sui registri, la RAM può avere fino a 2^{32} locazioni di memoria indirizzabili (ciascuna grande 1 byte = 8 bit), in altre parole la sua dimensione massima è di

$$2^{32}\,byte \simeq 4\,miliardi\,di\,byte \simeq 4\,GB$$

In un sistema simile, quindi, per valore di un puntatore è necessario e sufficiente usare un numero intero a 32 bit,
ovvero 4 byte (4 * 8 = 32), cioè 4 locazioni contigue di memoria. Ma allora a che serve indicare il **tipo** del puntatore?

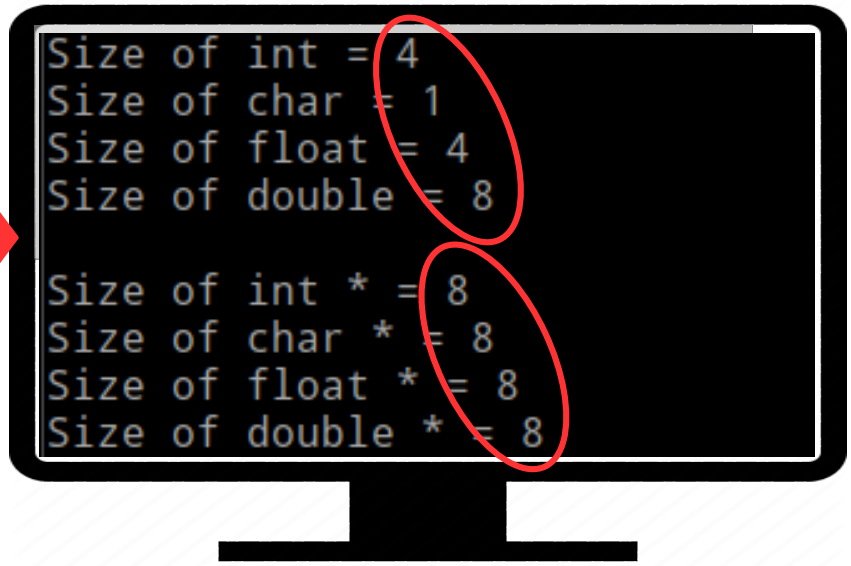
```
double * px = &x;
```



TIPI, PUNTATORI A TIPI, DIMENSIONE

```
printf("Size of int = %ld\n", sizeof(int ));
printf("Size of char = %ld\n", sizeof(char ));
printf("Size of float = %ld\n", sizeof(float ));
printf("Size of double = %ld\n", sizeof(double ));

printf("Size of int * = %ld\n", sizeof(int *));
printf("Size of char * = %ld\n", sizeof(char *));
printf("Size of float * = %ld\n", sizeof(float *));
printf("Size of double * = %ld\n", sizeof(double *));
```

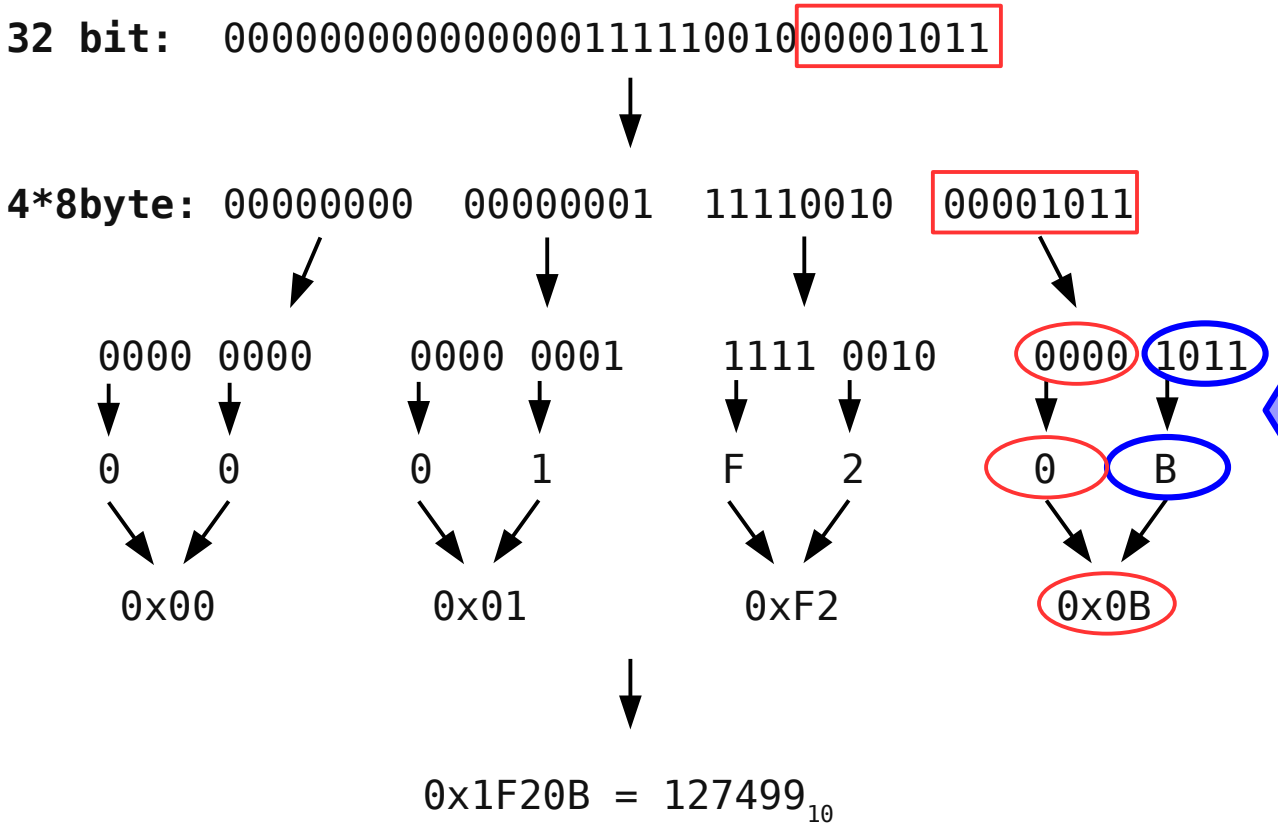


Possiamo usare sizeof() per conoscere la dimensione in byte dell'impronta in memoria di un certo tipo di dato. Per esempio possiamo stampare la dimensione dei tipi int, char, float e double, e poi dei rispettivi tipi “puntatore a int” (int *), “puntatore a char” (char *), “puntatore a float” (float *), “puntatore a double” (double *).

Eseguendo tale programma vediamo che mentre le dimensioni dei tipi int, char, float e double differiscono (come abbiamo illustrato nelle precedenti lezioni), la dimensione di tutti i diversi puntatori è sempre la stessa.

Ciascun puntatore richiede sempre 8 byte, cioè 8*8bit = 64 bit, perché tale è la misura di un indirizzo di memoria RAM in un sistema a 64 bit. Esso non rappresenta un numero intero, o un carattere, o un numero reale, ma sempre e soltanto **un indirizzo di memoria.**

PROMEMORIA: BIT, BYTE, BINARY, HEXADECIMAL



Binario	Esadecimale
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Nota: Più alta è la base (16 > 10 > 2), meno cifre servono per rappresentare un numero.

ARITMETICA DEI PUNTATORI

- Possiamo sommare e sottrarre valori interi da puntatori
- **Il tipo del puntatore serve a definire cosa succede quando si sommano o sottraggono valori interi da una variabile di tipo puntatore**
- Sommare un valore intero N a un puntatore significa spostarsi in avanti di N locazioni di memoria, ciascuna di dimensione necessaria a rappresentare il tipo della variabile puntatore stessa
- Se per esempio p è un puntatore di tipo double, che può puntare a una variabile double rappresentata da 8 byte, sommare un valore intero N significa spostarsi di N locazioni di memoria di dimensione necessaria a rappresentare un double, ovvero spostarsi di N*8 byte
- Sottrarre un intero N a un puntatore significa invece, analogamente, spostarsi indietro di N locazioni di dimensione necessaria a rappresentare il tipo del puntatore

PUNTATORE AD ARRAY

```
int a[10];
a[6] = 19;
int * p = a;
```

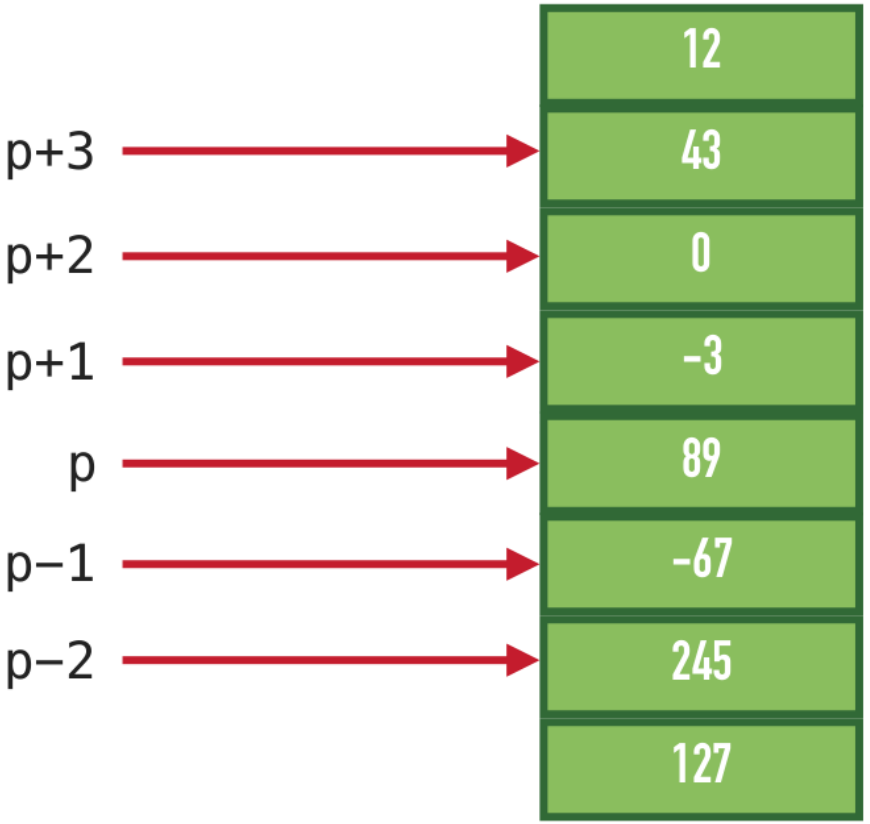
Notate che non si prende
"l'indirizzo di memoria di *a*"

Perché un array è rappresentato
da un indirizzo di memoria

`p[5]` Possiamo accedere agli elementi come se *p* fosse un array

p+1 sembra molto simile
a dire &p[1]...

...quindi magari
*(p+1) è come dire p[1]

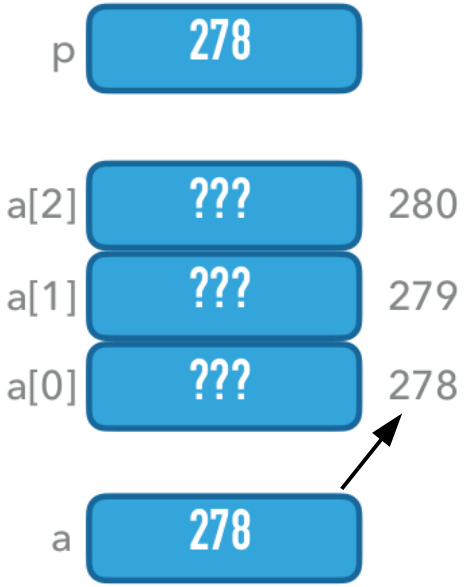


```
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char * argv[])
{
  int a[3];
  int * p = a;
  for (int i = 0; i < 3; i++) {
    *(p + i) = 42;
    printf("%d\n", a[i]);
  }
  exit(EXIT_SUCCESS);
}
```

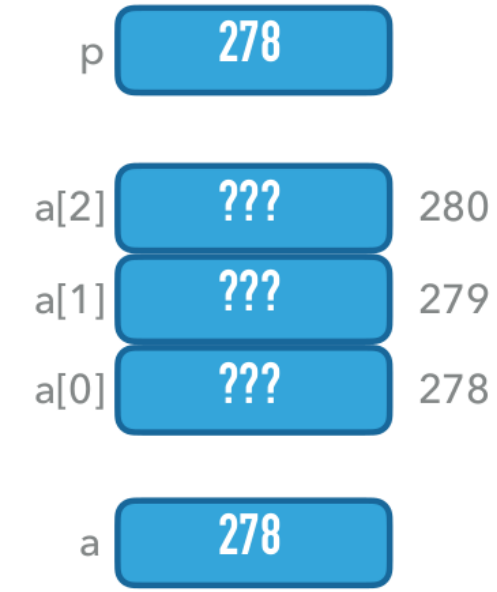
```
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char * argv[])
{
  int a[3];
  int * p = a;
  for (int i = 0; i < 3; i++) {
    *(p + i) = 42;
    printf("%d\n", a[i]);
  }
  exit(EXIT_SUCCESS);
}
```



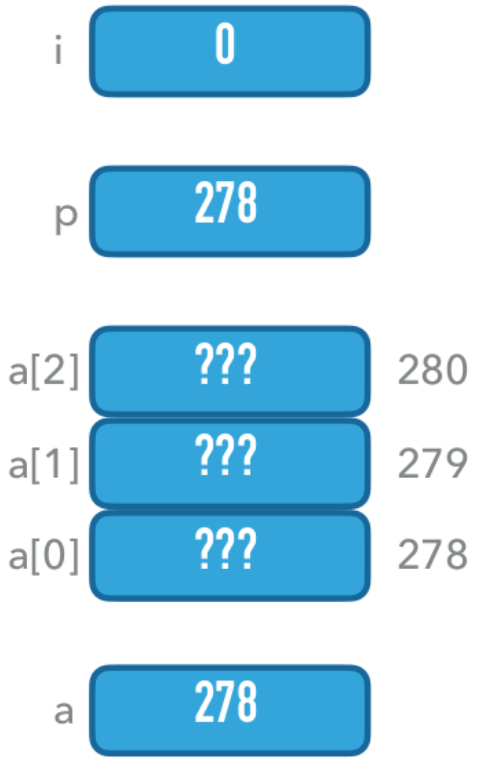
```
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char * argv[])
{
  int a[3];
  int * p = a;
  for (int i = 0; i < 3; i++) {
    *(p + i) = 42;
    printf("%d\n", a[i]);
  }
  exit(EXIT_SUCCESS);
}
```



```
#include <stdio.h>
#include <stdlib.h>

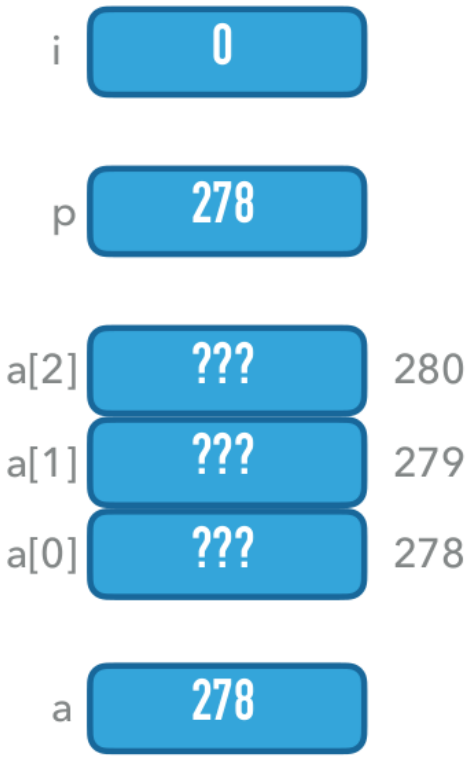
int main (int argc, char * argv[])
{
  int a[3];
  int * p = a;
  for (int i = 0; i < 3; i++) {
    *(p + i) = 42;
    printf("%d\n", a[i]);
  }
  exit(EXIT_SUCCESS);
}
```



```
#include <stdio.h>
#include <stdlib.h>

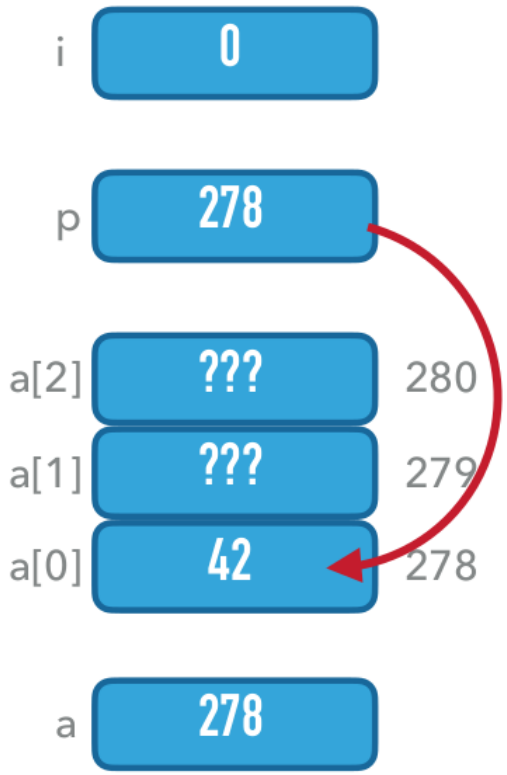
int main (int argc, char * argv[])
{
  int a[3];
  int * p = a;
  for (int i = 0; i < 3; i++) {
    *(p + i) = 42;
    printf("%d\n", a[i]);
  }
  exit(EXIT_SUCCESS);
}
```

0 < 3



```
#include <stdio.h>
#include <stdlib.h>

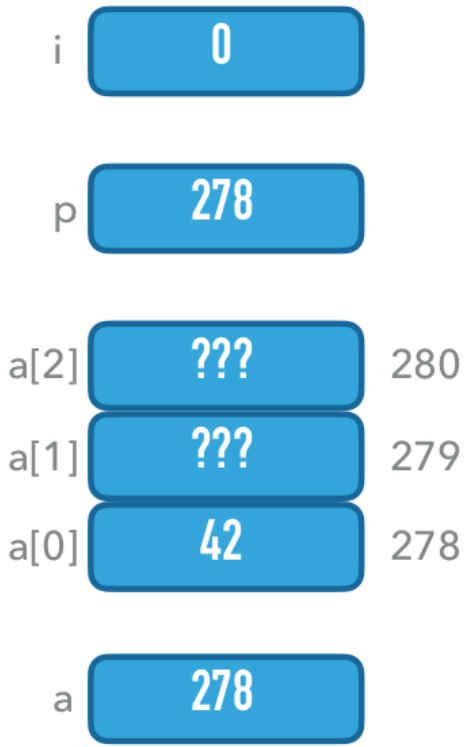
int main (int argc, char * argv[])
{
  int a[3];
  int * p = a;
  for (int i = 0; i < 3; i++) {
    *(p + i) = 42;
    printf("%d\n", a[i]);
  }
  exit(EXIT_SUCCESS);
}
```



```
#include <stdio.h>
#include <stdlib.h>

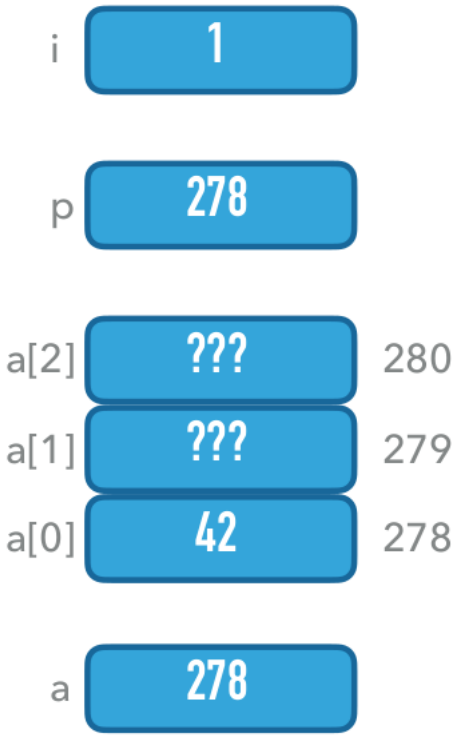
int main (int argc, char * argv[])
{
  int a[3];
  int * p = a;
  for (int i = 0; i < 3; i++) {
    *(p + i) = 42;
    printf("%d\n", a[i]);
  }
  exit(EXIT_SUCCESS);
}
```





```c
#include <stdio.h>
#include <stdlib.h>

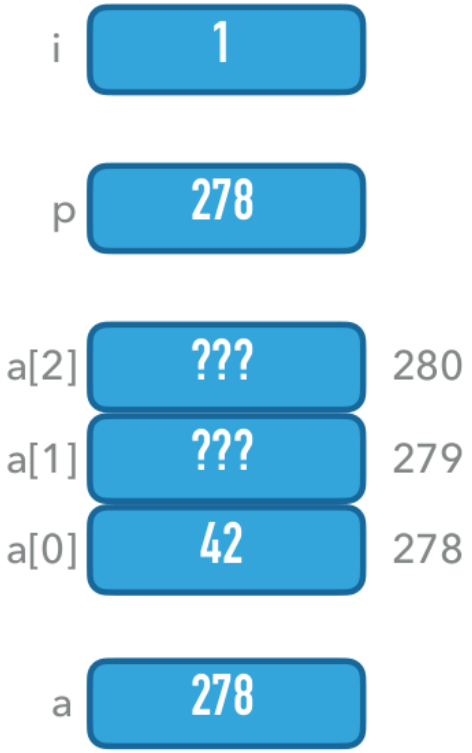
int main (int argc, char * argv[])
{
  int a[3];
  int * p = a;
  for (int i = 0; i < 3; i++) {
    *(p + i) = 42;
    printf("%d\n", a[i]);
  }
  exit(EXIT_SUCCESS);
}
```



```
#include <stdio.h>
#include <stdlib.h>

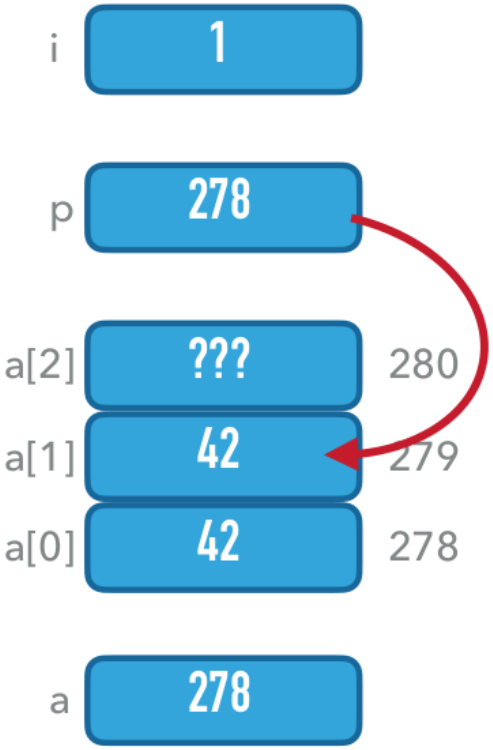
int main (int argc, char * argv[])
{
  int a[3];
  int * p = a;
  for (int i = 0; i < 3; i++) {
    *(p + i) = 42;
    printf("%d\n", a[i]);
  }
  exit(EXIT_SUCCESS);
}
```

1 < 3



```
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char * argv[])
{
  int a[3];
  int * p = a;
  for (int i = 0; i < 3; i++) {
    *(p + i) = 42;
    printf("%d\n", a[i]);
  }
  exit(EXIT_SUCCESS);
}
```



GESTIONE DINAMICA DELLA MEMORIA

- Negli esempi visti fin qui il numero di dati, per esempio il numero di elementi di un array, era fissato e noto a priori
- Naturalmente non è sempre questo il caso, e allora per situazioni più complesse dobbiamo capire come è divisa e utilizzabile in modo dinamico la memoria di un programma

STACK

In generale per *stack* (o pila) si intende una struttura di dati di “impilata”, di cui è accessibile solo un elemento, il primo della pila (per esempio un mazzo di carte dal quale non possiamo estrarre carte se non partendo sempre dalla prima in alto)

Quanto si parla di stack con riferimento a un tipo di gestione della memoria (ricordo che fisicamente la RAM sempre quella è, ciè che cambia è come la si utilizza), allora si chiama stack quel tipo (o area) di memoria usata in modo consecutivo e che abbiamo usato fino ad ora.

Ogni chiamata di funzione alloca un record di attivazione che contiene i parametri, le variabili locali, la cella in cui restituire l'uscita di una funzione

I record di attivazione sono aggiunti e rimossi come una “pila di piatti”

STACK E RECORD DI ATTIVAZIONE

```
void g()
{
  //…
}

void f()
{
  g();
}

int main(int argc, char* argv[])
{
  f();
}
```

STACK DI ATTIVAZIONE
DELLA FUNZIONE MAIN

STECK E RECORD DI ATTIVAZIONE




```
void g()
{
  //…
}

void f()
{
  g();
}

int main(int argc, char* argv[])
{
  f();
}
```

STACK




```
void g()
{
  //…
}

void f()
{
  g();
}

int main(int argc, char* argv[])
{
  f();
}
```

STACK E RECORD DI ATTIVAZIONE




```
void g()
{
  //…
}

void f()
{
  g();
}

int main(int argc, char* argv[])
{
  f();
}
```

STACK E RECORD DI ATTIVAZIONE

```
void g()
{
  //…
}

void f()
{
  g();
}

int main(int argc, char* argv[])
{
  f();
}
```

STACK DI ATTIVAZIONE
DELLA FUNZIONE MAIN

STACK: VANTAGGI E LIMITAZIONI

Le variabili locali cessano di esistere quando la funzione termina

Possiamo modificare tramite puntatori solo valori allocati da funzioni più “in profondità” nello stack

In generale non abbiamo controllo su quando la memoria viene allocata e deallocata. D'altra parte questo è un vantaggio se non siamo noi a dovercene occupare

STACK E HEAP



Si può gestire la memoria heap in modo che questa non sia necessariamente impilata e consecutiva, come lo stack

In questo caso dobbiamo richiedere esplicitamente l'allocazione della memoria

Possiamo decidere durante l'esecuzione quanta memoria allocare

Ma: dobbiamo occuparci noi di restituire la memoria una volta che non è più necessaria

HEAP: COME RICHIEDERNE L'USO

Usiamo una funzione che si occupa di richiedere la memoria: **`malloc`**

Dobbiamo indicare la dimensione: per farlo dobbiamo dire quanti dati di un certo tipo dovrà contenere (useremo per esempio **`N * sizeof(tipo)`**)

La funzione malloc() **restituisce un puntatore** alla memoria allocata, cioè al primo byte della sequenza allocata

Una volta che non ci serve più la memoria la liberiamo chiamando **`free`**

ALLOCAZIONE DINAMICA CON MALLOC

Puntatore

Puntatore in cui salviamo l'indirizzo di memoria ritornato da malloc.

Dimensione

Quanto spazio di memoria richiedere al sistema, in byte

```
puntatore = malloc(dimensione);
```

```
int * a = malloc(sizeof(int));
```

Allochiamo sizeof(int) byte

```
float * b = malloc(sizeof(float));
```

Allochiamo sizeof(float) byte

```
struct punto * c = malloc(sizeof(struct punto));
```

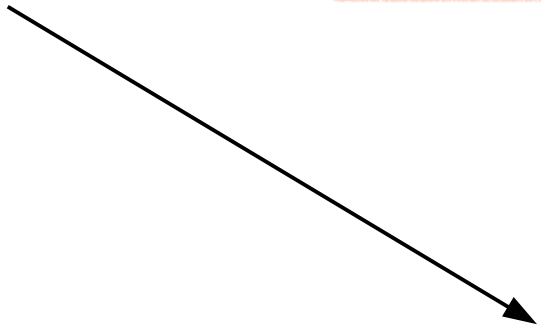
Allochiamo sizeof(struct punto) byte

ALLOCARE UN ARRAY CON MALLOC

```
int * a = malloc(sizeof(int));
```

Qui abbiamo allocato la memoria necessaria a contenere un intero.
E se volessimo 10 interi?

```
int * a = malloc(10 * sizeof(int));
```



Dimensione

10 volte la dimensione di un intero, l'allocazione di questo spazio è contigua in memoria

Accediamo agli elementi come: `a[5]` oppure `*(a+5)`

NOTE SU MALLOC

```
int * a = (int *)malloc(sizeof(int));
```

A volte si può voler fare un cast per forzare il puntatore ad essere del tipo corretto (int * in questo caso)

```
int * a = (int *)malloc(sizeof(int));
if (a == NULL) {
  // errore: allocazione di memoria non riuscita
}
```

Se l'allocazione di memoria fallisce viene ritornato il valore NULL, in quel caso la memoria NON è stata allocata

MALLOC: ESEMPIO

```
#include <stdio.h>
#include <stdlib.h>

int * crea_array(int dim, int valore)
{
  int * v = malloc(dim * sizeof(int));
  for (int i = 0; i < dim; i++) {
    v[i] = valore;
  }
  return v;
}

int main (int argc, char * argv[])
{
  int * p = crea_array(4, 2);
  printf("%d\n", p[3]);
  exit(EXIT_SUCCESS);
}
```

MALLOC: ESEMPIO

```
#include <stdio.h>
#include <stdlib.h>

int * crea_array(int dim, int valore)
{
  int * v = malloc(dim * sizeof(int));
  for (int i = 0; i < dim; i++) {
    v[i] = valore;
  }
  return v;
}

int main (int argc, char * argv[])
{
  int * p = crea_array(4, 2);
  printf("%d\n", p[3]);
  exit(EXIT_SUCCESS);
}
```

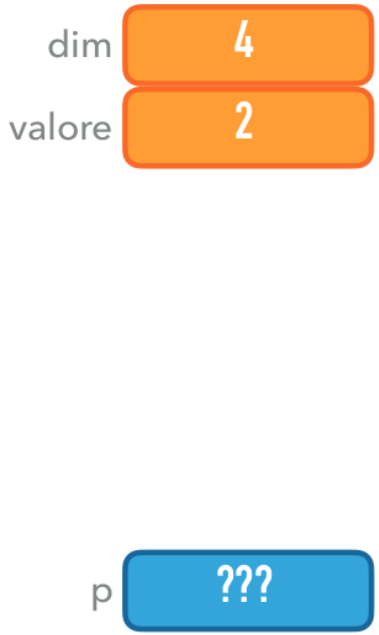


MALLOC: ESEMPIO

```
#include <stdio.h>
#include <stdlib.h>

int * crea_array(int dim, int valore)
{
  int * v = malloc(dim * sizeof(int));
  for (int i = 0; i < dim; i++) {
    v[i] = valore;
  }
  return v;
}

int main (int argc, char * argv[])
{
  int * p = crea_array(4, 2);
  printf("%d\n", p[3]);
  exit(EXIT_SUCCESS);
}
```

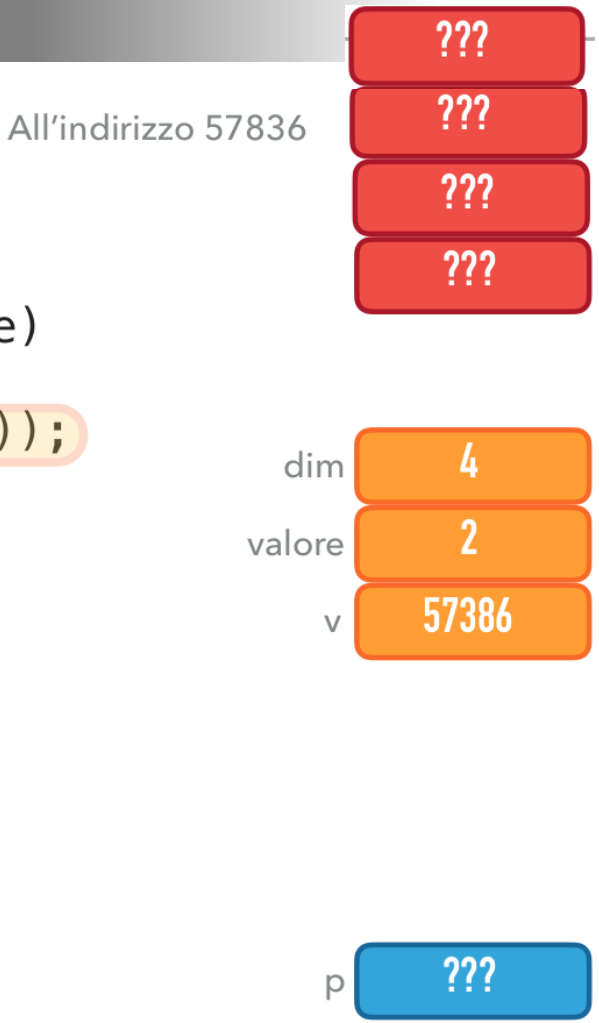


MALLOC: ESEMPIO

```
#include <stdio.h>
#include <stdlib.h>

int * crea_array(int dim, int valore)
{
  int * v = malloc(dim * sizeof(int));
  for (int i = 0; i < dim; i++) {
    v[i] = valore;
  }
  return v;
}

int main (int argc, char * argv[])
{
  int * p = crea_array(4, 2);
  printf("%d\n", p[3]);
  exit(EXIT_SUCCESS);
}
```

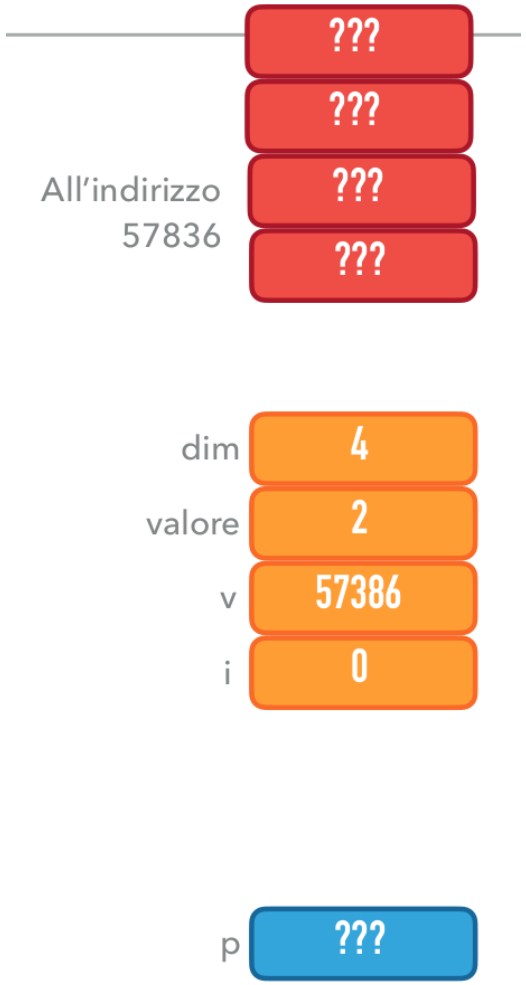


MALLOC: ESEMPIO

```
#include <stdio.h>
#include <stdlib.h>

int * crea_array(int dim, int valore)
{
  int * v = malloc(dim * sizeof(int));
  for (int i = 0; i < dim; i++) {
    v[i] = valore;
  }
  return v;
}

int main (int argc, char * argv[])
{
  int * p = crea_array(4, 2);
  printf("%d\n", p[3]);
  exit(EXIT_SUCCESS);
}
```

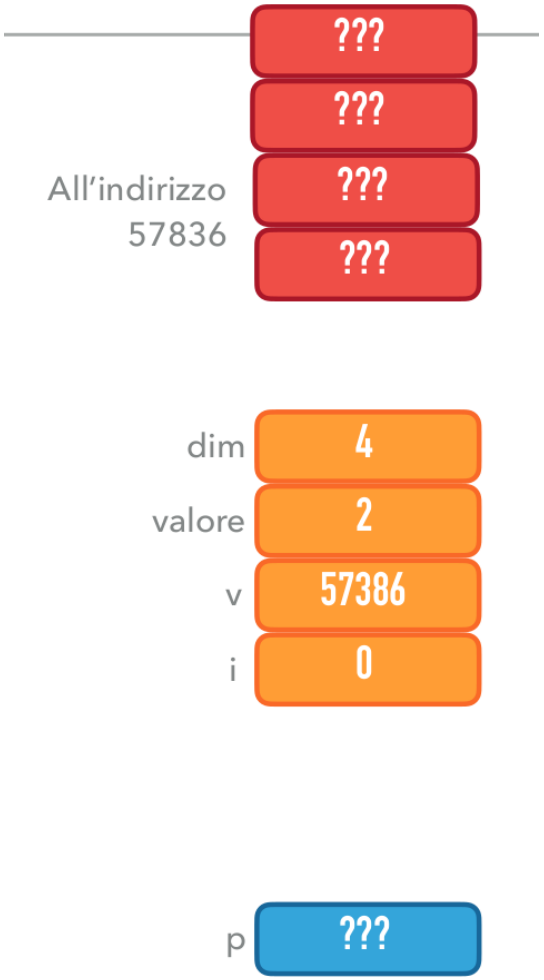


MALLOC: ESEMPIO

```
#include <stdio.h>
#include <stdlib.h>

int * crea_array(int dim, int valore)
{
  int * v = malloc(dim * sizeof(int));
  for (int i = 0; i < dim; i++) { 0<4
    v[i] = valore;
  }
  return v;
}

int main (int argc, char * argv[])
{
  int * p = crea_array(4, 2);
  printf("%d\n", p[3]);
  exit(EXIT_SUCCESS);
}
```

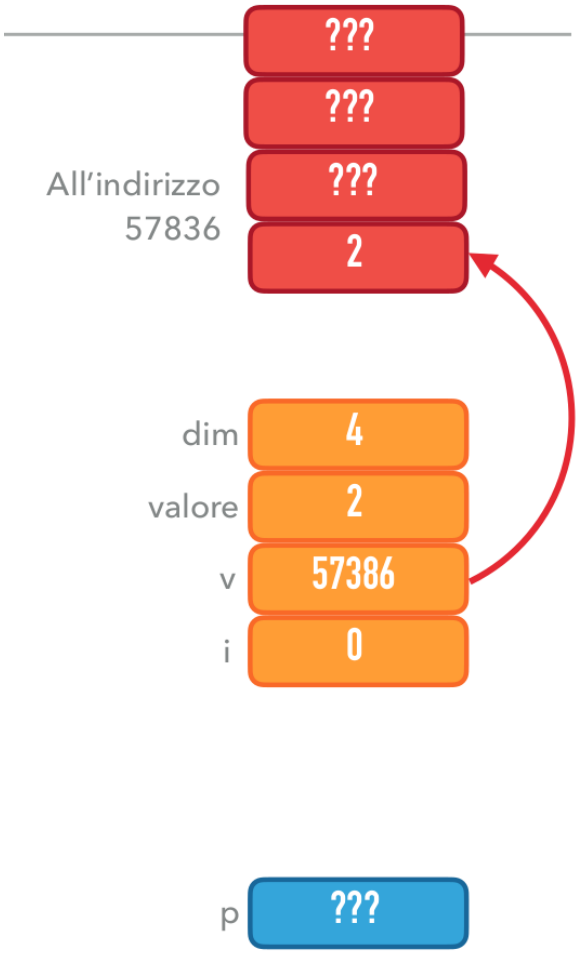


MALLOC: ESEMPIO

```c
#include <stdio.h>
#include <stdlib.h>

int * crea_array(int dim, int valore)
{
  int * v = malloc(dim * sizeof(int));
  for (int i = 0; i < dim; i++) {
    v[i] = valore;
  }
  return v;
}

int main (int argc, char * argv[])
{
  int * p = crea_array(4, 2);
  printf("%d\n", p[3]);
  exit(EXIT_SUCCESS);
}
```

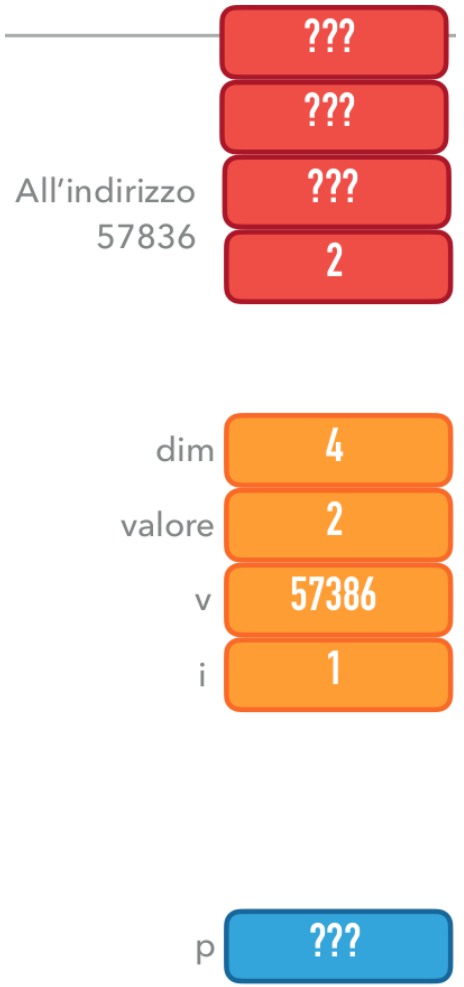


MALLOC: ESEMPIO

```c
#include <stdio.h>
#include <stdlib.h>

int * crea_array(int dim, int valore)
{
  int * v = malloc(dim * sizeof(int));
  for (int i = 0; i < dim; i++) {
    v[i] = valore;
  }
  return v;
}

int main (int argc, char * argv[])
{
  int * p = crea_array(4, 2);
  printf("%d\n", p[3]);
  exit(EXIT_SUCCESS);
}
```

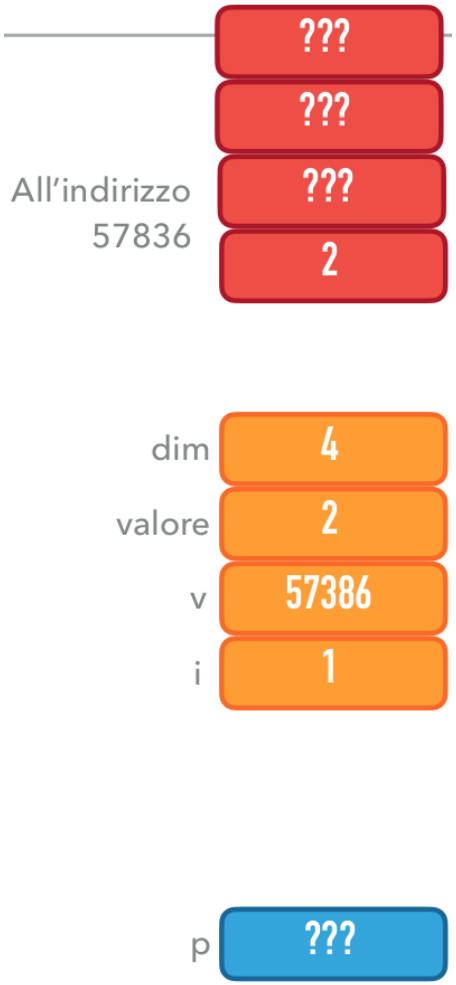


MALLOC: ESEMPIO

```
#include <stdio.h>
#include <stdlib.h>

int * crea_array(int dim, int valore)
{
  int * v = malloc(dim * sizeof(int));
  for (int i = 0; i < dim; i++) { 1<4
    v[i] = valore;
  }
  return v;
}

int main (int argc, char * argv[])
{
  int * p = crea_array(4, 2);
  printf("%d\n", p[3]);
  exit(EXIT_SUCCESS);
}
```

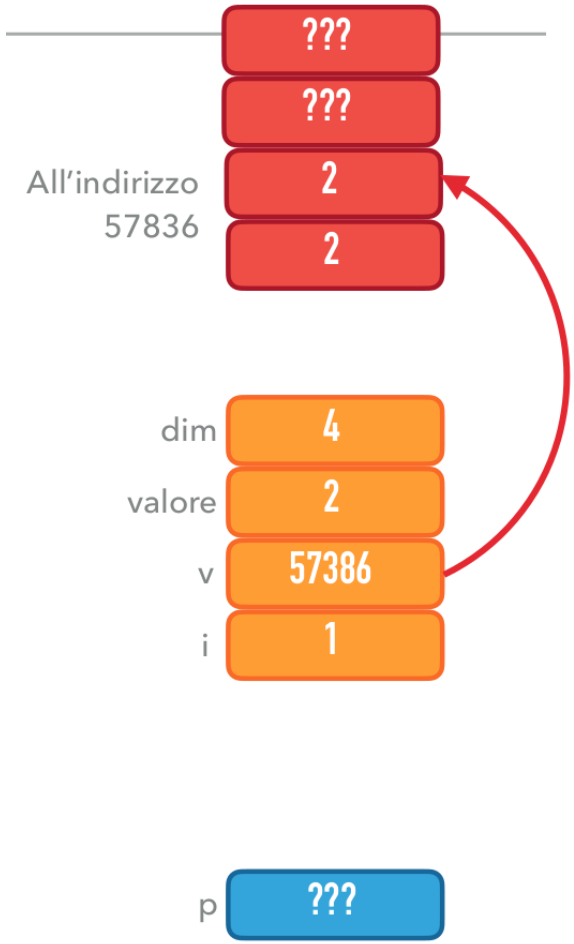


MALLOC: ESEMPIO

```
#include <stdio.h>
#include <stdlib.h>

int * crea_array(int dim, int valore)
{
  int * v = malloc(dim * sizeof(int));
  for (int i = 0; i < dim; i++) {
    v[i] = valore;
  }
  return v;
}

int main (int argc, char * argv[])
{
  int * p = crea_array(4, 2);
  printf("%d\n", p[3]);
  exit(EXIT_SUCCESS);
}
```

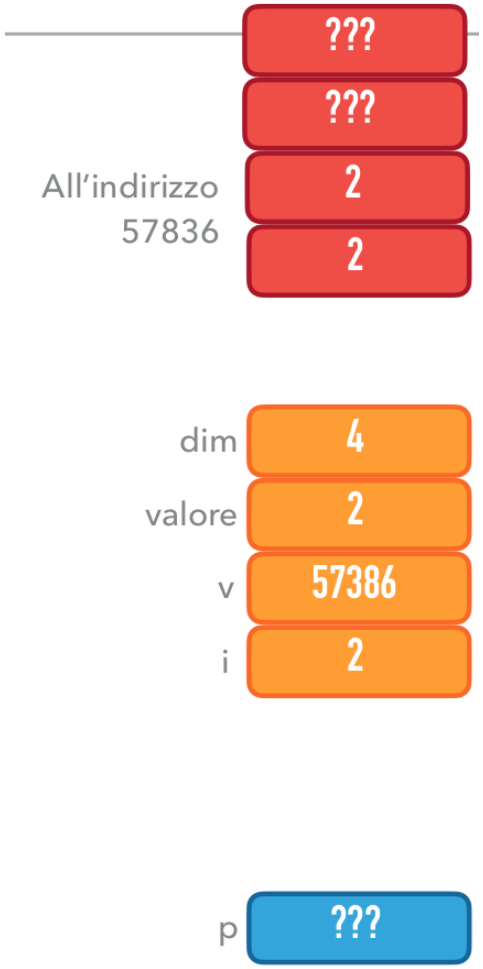


MALLOC: ESEMPIO

```
#include <stdio.h>
#include <stdlib.h>

int * crea_array(int dim, int valore)
{
  int * v = malloc(dim * sizeof(int));
  for (int i = 0; i < dim; i++) {
    v[i] = valore;
  }
  return v;
}

int main (int argc, char * argv[])
{
  int * p = crea_array(4, 2);
  printf("%d\n", p[3]);
  exit(EXIT_SUCCESS);
}
```

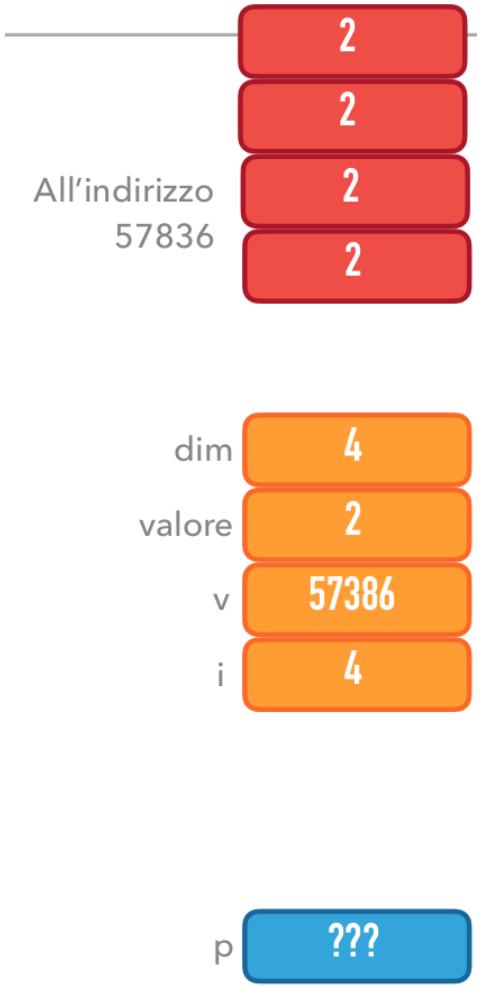


DOPO DIVERSI PASSI…

```
#include <stdio.h>
#include <stdlib.h>

int * crea_array(int dim, int valore)
{
  int * v = malloc(dim * sizeof(int));
  for (int i = 0; i < dim; i++) {   4<4
    v[i] = valore;
  }
  return v;
}

int main (int argc, char * argv[])
{
  int * p = crea_array(4, 2);
  printf("%d\n", p[3]);
  exit(EXIT_SUCCESS);
}
```

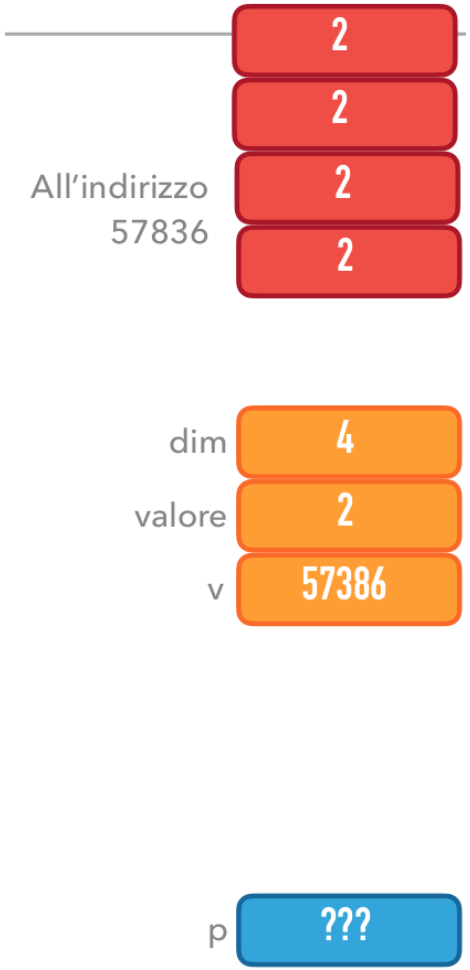


DOPO DIVERSI PASSI…

```
#include <stdio.h>
#include <stdlib.h>

int * crea_array(int dim, int valore)
{
  int * v = malloc(dim * sizeof(int));
  for (int i = 0; i < dim; i++) {
    v[i] = valore;
  }
  return v;
}

int main (int argc, char * argv[])
{
  int * p = crea_array(4, 2);
  printf("%d\n", p[3]);
  exit(EXIT_SUCCESS);
}
```

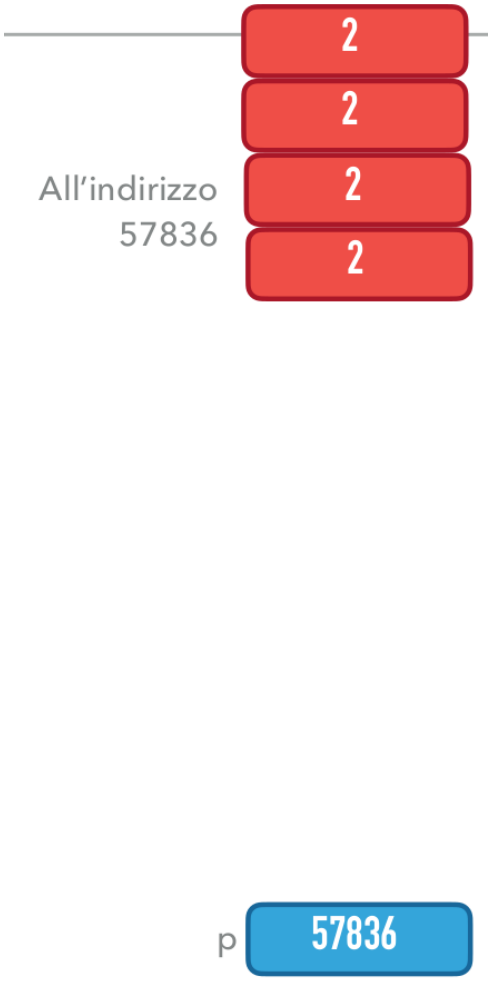


MALLOC: ESEMPIO

```
#include <stdio.h>
#include <stdlib.h>

int * crea_array(int dim, int valore)
{
  int * v = malloc(dim * sizeof(int));
  for (int i = 0; i < dim; i++) {
    v[i] = valore;
  }
  return v;
}

int main (int argc, char * argv[])
{
  int * p = crea_array(4, 2);
  printf("%d\n", p[3]);
  exit(EXIT_SUCCESS);
}
```

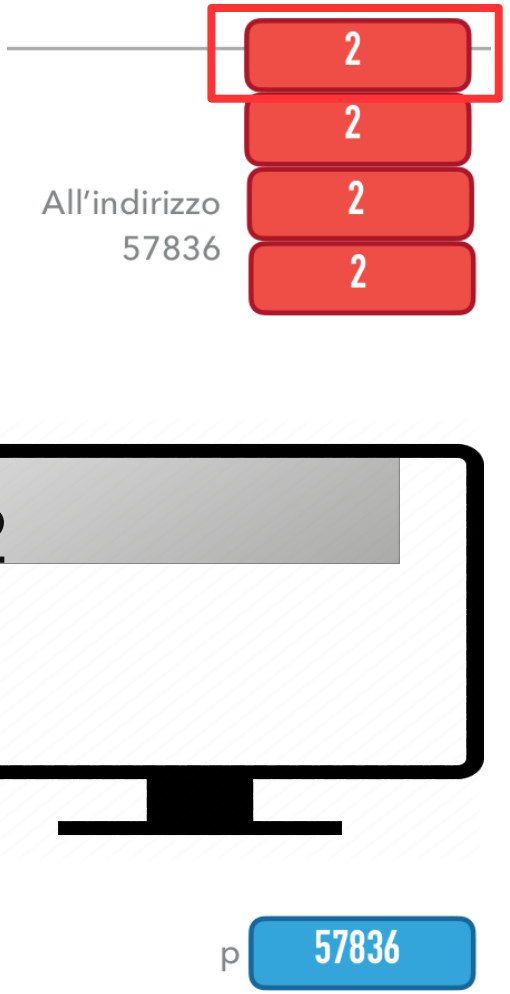


MALLOC: ESEMPIO

```
#include <stdio.h>
#include <stdlib.h>

int * crea_array(int dim, int valore)
{
  int * v = malloc(dim * sizeof(int));
  for (int i = 0; i < dim; i++) {
    v[i] = valore;
  }
  return v;
}

int main (int argc, char * argv[])
{
  int * p = crea_array(4, 2)
  printf("%d\n", p[3]);
  exit(EXIT_SUCCESS);
}
```



FREE: DEALLOCARE LA MEMORIA

```
int main (int argc, char * argv[])
{
  int * p = crea_array(4, 2);
  printf("%d\n", p[3]);
  free(p);
  exit(EXIT_SUCCESS);
}
```

Attenzione

utilizzare la memoria dopo averla deallocata è un errore!

CAVEAT

Ricordarsi di allocare la quantità giusta di memoria, usando sizeof con il tipo corretto

Se “perdete” un puntatore ad un’area di memoria allocata con malloc quella memoria non verrà liberata, si ha un *memory leak*

Ricordatevi se la memoria è gestita in automatico (stack) o se invece l'avete richiesta voi con malloc e quindi rimane finché non la deallocate (heap)

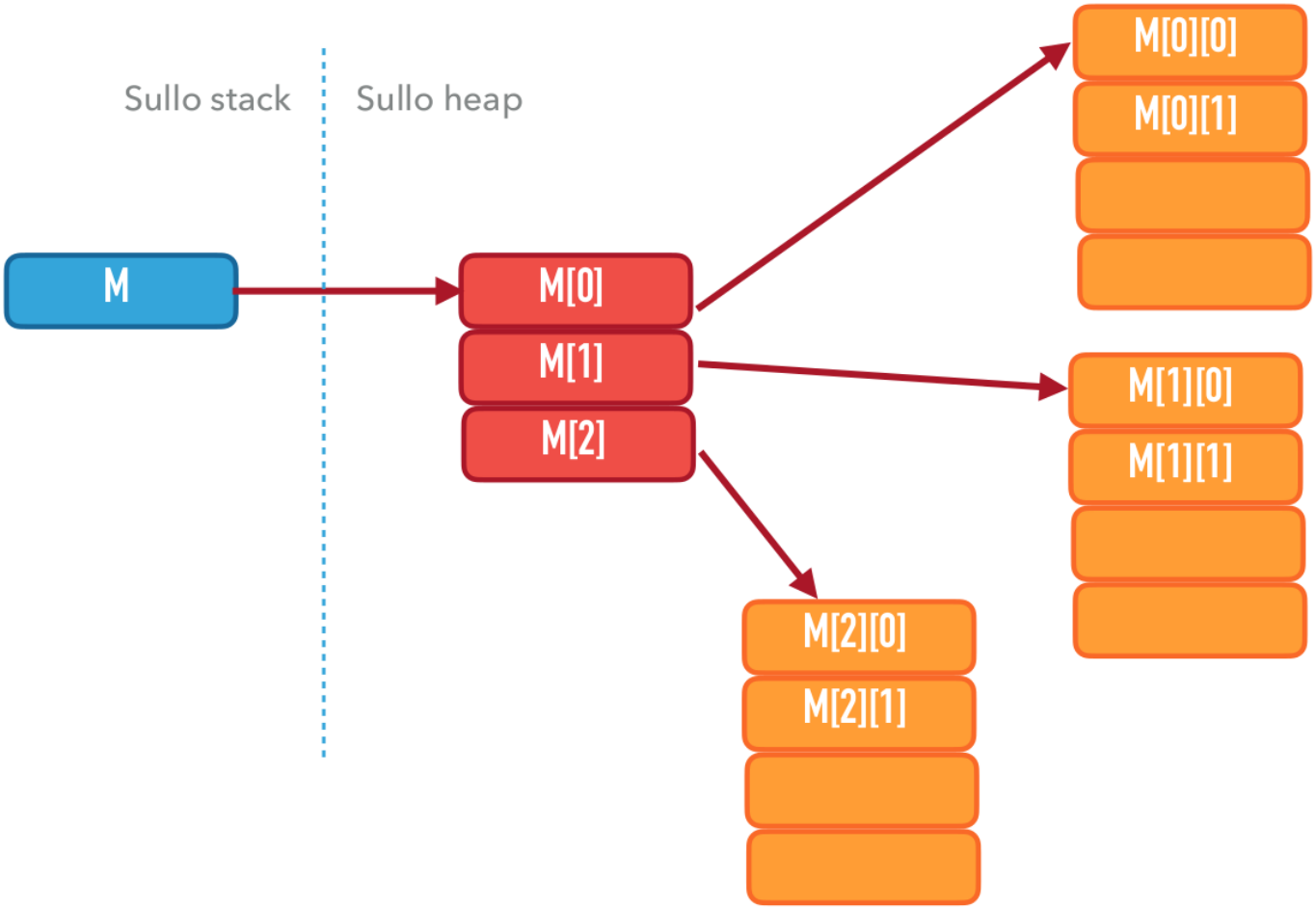
PUNTATORI A PUNTATORI E MATRICI CON MALLOC

```
int righe = 3;
int colonne = 4;
int ** M; // puntatore a puntatore a int

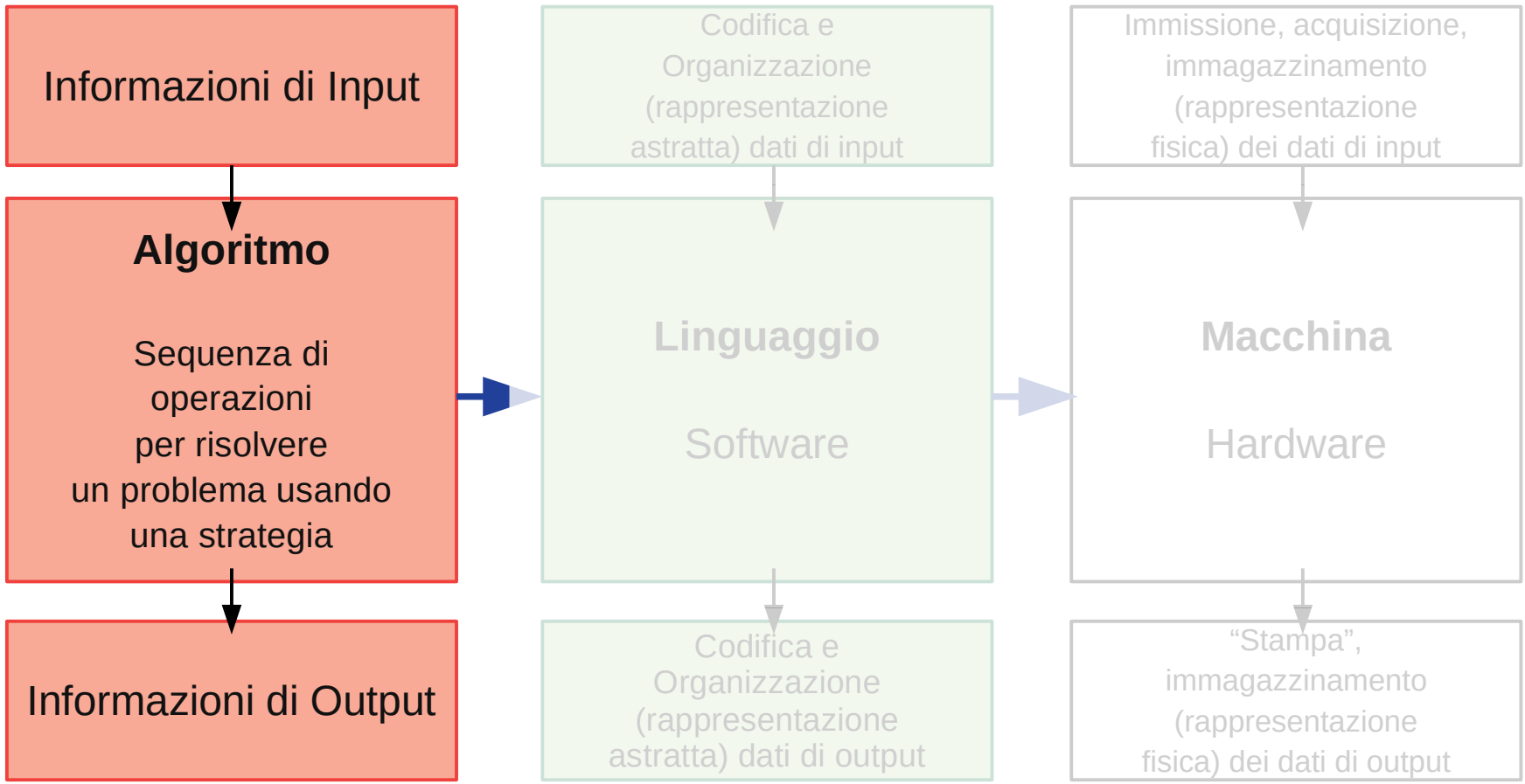

/* allochiamo un array di 3 puntatori a int */
M = (int **) malloc(righe * sizeof(int *));


/* ognuno di quei puntatori "punta" ad un
   array di 4 interi */
for (int i = 0; i < righe; i++) {
   M[i] = (int *) malloc(colonne * sizeof(int));
}
```

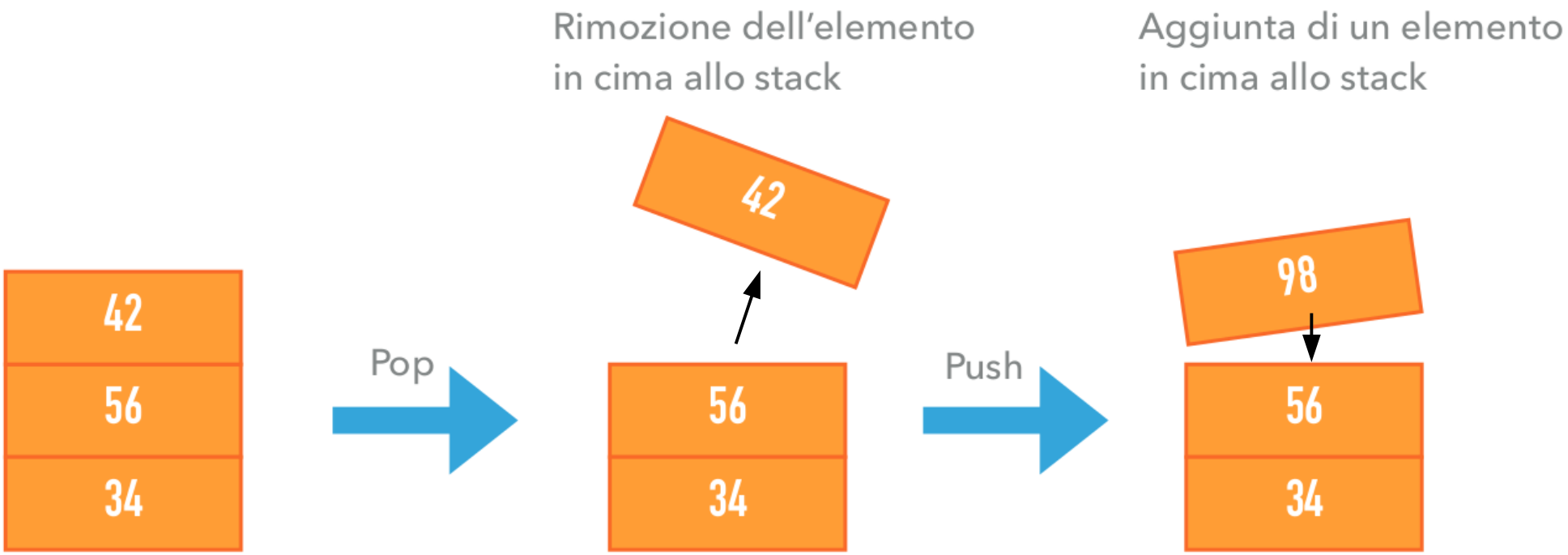
PUNTATORI A PUNTATORI E MATRICI CON MALLOC



STACK (O PILA) DI DATI



STACK (O PILE) COME STRUTTURE DI DATI: UN ESEMPIO



NOTAZIONE POLACCA INVERSA

$$(9 \times 3) + (5/4)$$

Proviamo a mettere gli operatori dopo gli operandi

$$(9\ 3\ \times)\ (5\ 4\ /)\ +$$

Ora l'ordine di applicazione degli operatori è ben specificato.
Possiamo eliminare le parentesi:

$$9\ 3\ \times\ 5\ 4\ /\ +$$

Definizione di **arietà** di un operatore: numero di operandi che richiede

NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +

Posizione della lettura



Se il valore che leggo è un numero
  lo inserisco nello stack
Altrimenti se è una operazione
  ottengo i due operandi:
  x = pop()
  y = pop()
  calcolo *z = y op x*
  e lo inserisco nello stack

Stack (vuoto al momento)

NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +

Posizione della lettura

Se il valore che leggo è un numero
  lo inserisco nello stack
Altrimenti se è una operazione
  ottengo i due operandi:
  x = pop()
  y = pop()
  calcolo *z = y op x*
  e lo inserisco nello stack

9

Stack

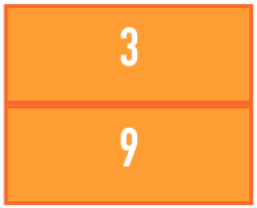
NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +

Posizione della lettura



Se il valore che leggo è un numero
  lo inserisco nello stack
Altrimenti se è una operazione
  ottengo i due operandi:
  x = pop()
  y = pop()
  calcolo *z = y op x*
  e lo inserisco nello stack



Stack

NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +

Posizione della lettura

x

3

Se il valore che leggo è un numero
lo inserisco nello stack
Altrimenti se è una operazione
ottengo i due operandi:
x = pop()
y = pop()
calcolo *z = y op x*
e lo inserisco nello stack

9

Stack

NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +

Posizione della lettura

x 3

y 9

Se il valore che leggo è un numero
  lo inserisco nello stack
Altrimenti se è una operazione
  ottengo i due operandi:
  x = pop()
  y = pop()
  calcolo *z = y op x*
  e lo inserisco nello stack

Stack (vuoto al momento)

NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +

Posizione della lettura



Se il valore che leggo è un numero
lo inserisco nello stack
Altrimenti se è una operazione
ottengo i due operandi:
x = pop()
y = pop()
calcolo *z = y op x*
e lo inserisco nello stack

Stack (vuoto al momento)

NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +

Posizione della lettura

Se il valore che leggo è un numero
  lo inserisco nello stack
Altrimenti se è una operazione
  ottengo i due operandi:
  x = pop()
  y = pop()
  calcolo *z = y op x*
  e lo inserisco nello stack

27

Stack

NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +

Posizione della lettura



Se il valore che leggo è un numero
  lo inserisco nello stack
Altrimenti se è una operazione
  ottengo i due operandi:
  x = pop()
  y = pop()
  calcolo *z = y op x*
  e lo inserisco nello stack



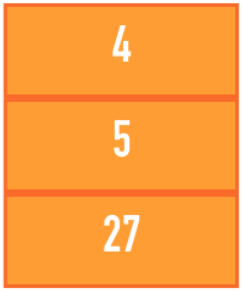
Stack

NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +

Posizione della lettura

Se il valore che leggo è un numero
  lo inserisco nello stack
Altrimenti se è una operazione
  ottengo i due operandi:
  x = pop()
  y = pop()
  calcolo *z = y op x*
  e lo inserisco nello stack



Stack

NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +

Posizione della lettura

x 4

5

27

Stack

Se il valore che leggo è un numero
lo inserisco nello stack
Altrimenti se è una operazione
ottengo i due operandi:
x = pop()
y = pop()
calcolo *z = y op x*
e lo inserisco nello stack

NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +

Posizione della lettura



x 4

y 5

27

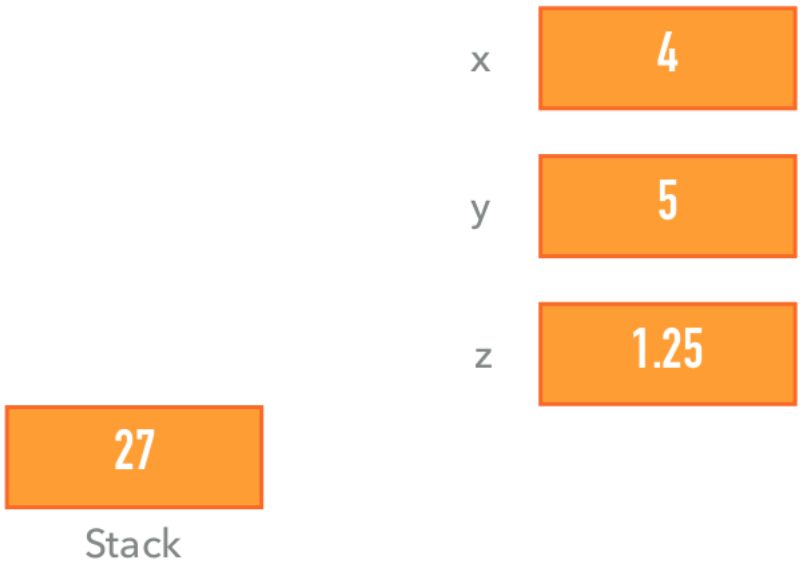
Stack

Se il valore che leggo è un numero
lo inserisco nello stack
Altrimenti se è una operazione
ottengo i due operandi:
x = pop()
y = pop()
calcolo *z = y op x*
e lo inserisco nello stack

NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +

Posizione della lettura



Se il valore che leggo è un numero
lo inserisco nello stack
Altrimenti se è una operazione
ottengo i due operandi:
x = pop()
y = pop()
calcolo *z = y op x*
e lo inserisco nello stack

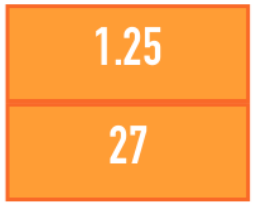
NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +

Posizione della lettura



Se il valore che leggo è un numero
  lo inserisco nello stack
Altrimenti se è una operazione
  ottengo i due operandi:
  x = pop()
  y = pop()
  calcolo *z = y op x*
  e lo inserisco nello stack



Stack

NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +

Posizione della lettura

x

1.25

Se il valore che leggo è un numero
  lo inserisco nello stack
Altrimenti se è una operazione
  ottengo i due operandi:
  x = pop()
  y = pop()
  calcolo *z = y op x*
  e lo inserisco nello stack

27

Stack

NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +

Posizione della lettura



x 1.25

y 27

Se il valore che leggo è un numero
  lo inserisco nello stack
Altrimenti se è una operazione
  ottengo i due operandi:
  x = pop()
  y = pop()
  calcolo *z = y op x*
  e lo inserisco nello stack

Stack (vuoto al momento)

NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +

Posizione della lettura



x 1.25

y 27

z 28.25

Stack (vuoto al momento)

Se il valore che leggo è un numero
  lo inserisco nello stack
Altrimenti se è una operazione
  ottengo i due operandi:
  x = pop()
  y = pop()
  calcolo *z = y op x*
  e lo inserisco nello stack

NOTAZIONE POLACCA INVERSA E STACK

9 3 × 5 4 / +



Posizione della lettura

Se il valore che leggo è un numero
  lo inserisco nello stack
Altrimenti se è una operazione
  ottengo i due operandi:
  x = pop()
  y = pop()
  calcolo *z = y op x*
  e lo inserisco nello stack

Finita la lettura il risultato è in cima allo stack



Stack